ECOMMERCE ORDER MANAGEMENT SYSTEM

Relational Database Design, CRUD Operations, Advanced Joins, Window Functions, CASE Expressions & Subqueries

Student Name	Priyal Patel
Course & Subject	SQL & Advanced Relational Database Management System (RDBMS)
DBMS Platform	MySQL Server & phpMyAdmin (Apache / XAMPP)
Project Directory / Script	Ecommerce Management (ecommerce_management.sql)

Executive Summary & Overview

The E-Commerce Order Management System is a comprehensive relational database solution developed in MySQL. The project models an enterprise e-commerce platform consisting of 7 interconnected entities: Categories, Products, Customers, Orders, Order_Items, Payments, and Shipping. The implementation covers complete schema definition (DDL), sample data seeding (DML), and 12 structured practical tasks demonstrating CRUD operations, filtering clauses (WHERE, HAVING, LIMIT), logical operators (AND, OR, NOT), sorting & grouping (ORDER BY, GROUP BY), aggregate analytics (SUM, AVG, COUNT, MIN, MAX), Primary and Foreign Key constraint management, relational Joins (INNER, LEFT, Multi-table), nested subqueries, date & time extraction, string manipulation, window ranking & running totals, and dynamic CASE expressions.

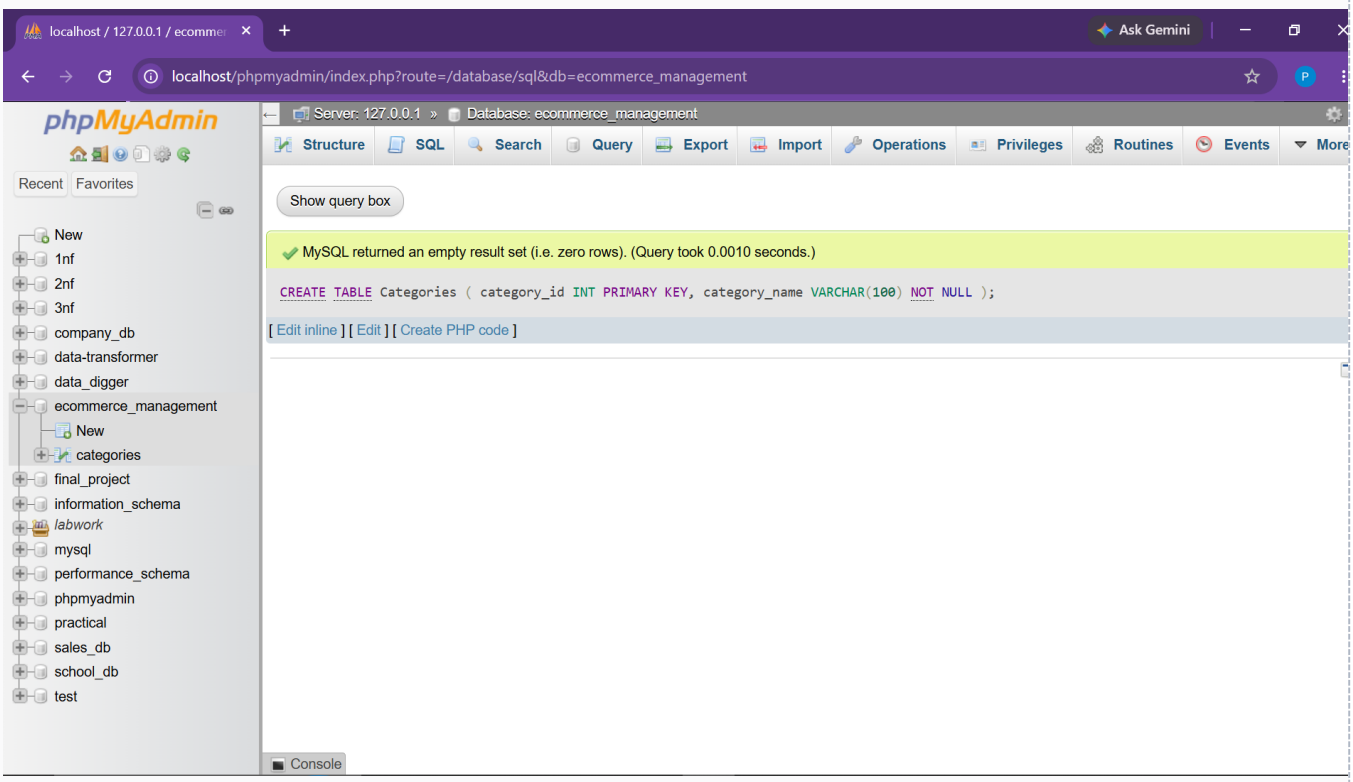
Each subsequent page of this report presents an individual SQL task with its exact query implementation, a dedicated full-width screenshot container for execution results in phpMyAdmin, and a concise analytical observation.

1. DATABASE SCHEMA & TABLE CREATION (DDL)

• 1.1 Create Categories Table

Objective: Define primary category classification table with CategoryID as primary key.

```
CREATE TABLE Categories (
    category_id INT PRIMARY KEY,
    category_name VARCHAR(100) NOT NULL
);
```

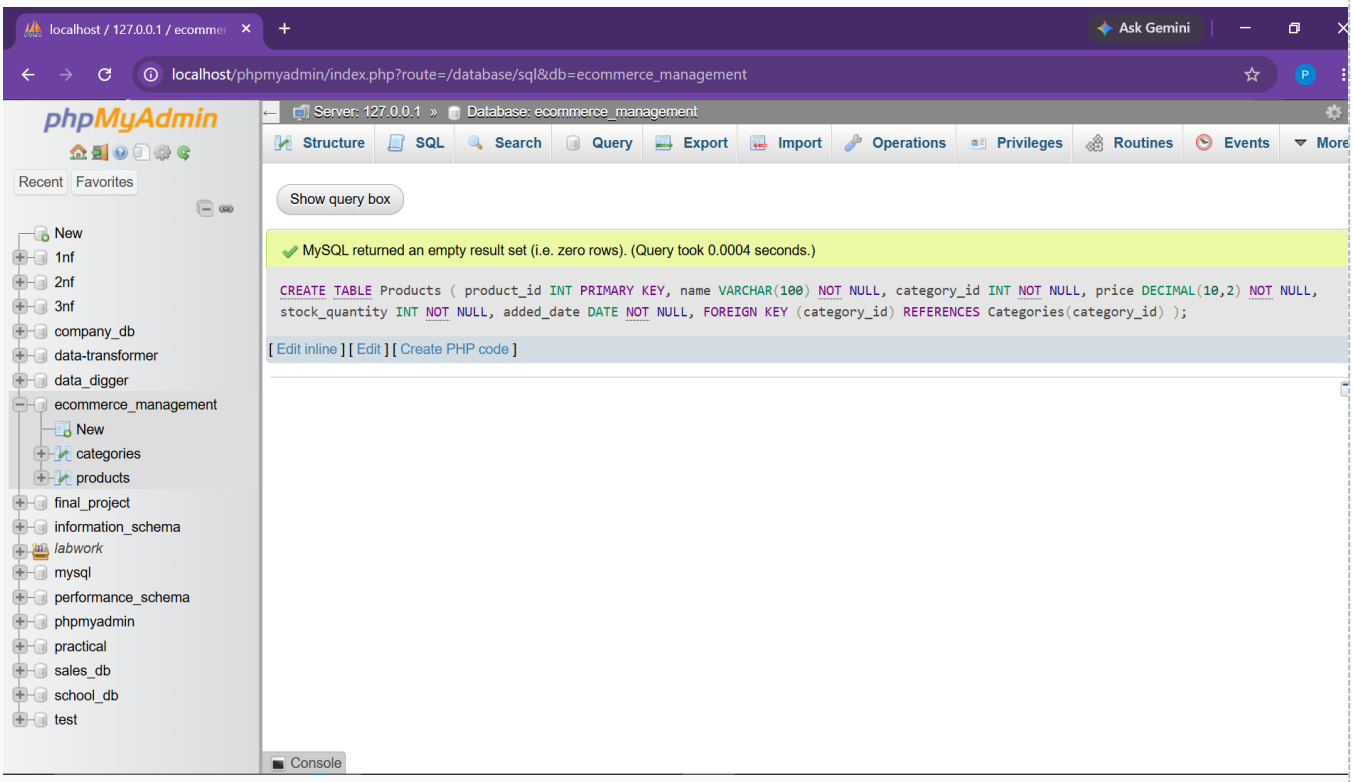


Observation: Categories table created successfully with category_id as the primary key.

- **1.2 Create Products Table with Foreign Key**

Objective: Establish products entity referencing parent Categories table.

```
CREATE TABLE Products (
    product_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    category_id INT NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    stock_quantity INT NOT NULL,
    added_date DATE NOT NULL,
    FOREIGN KEY (category_id) REFERENCES Categories(category_id)
);
```

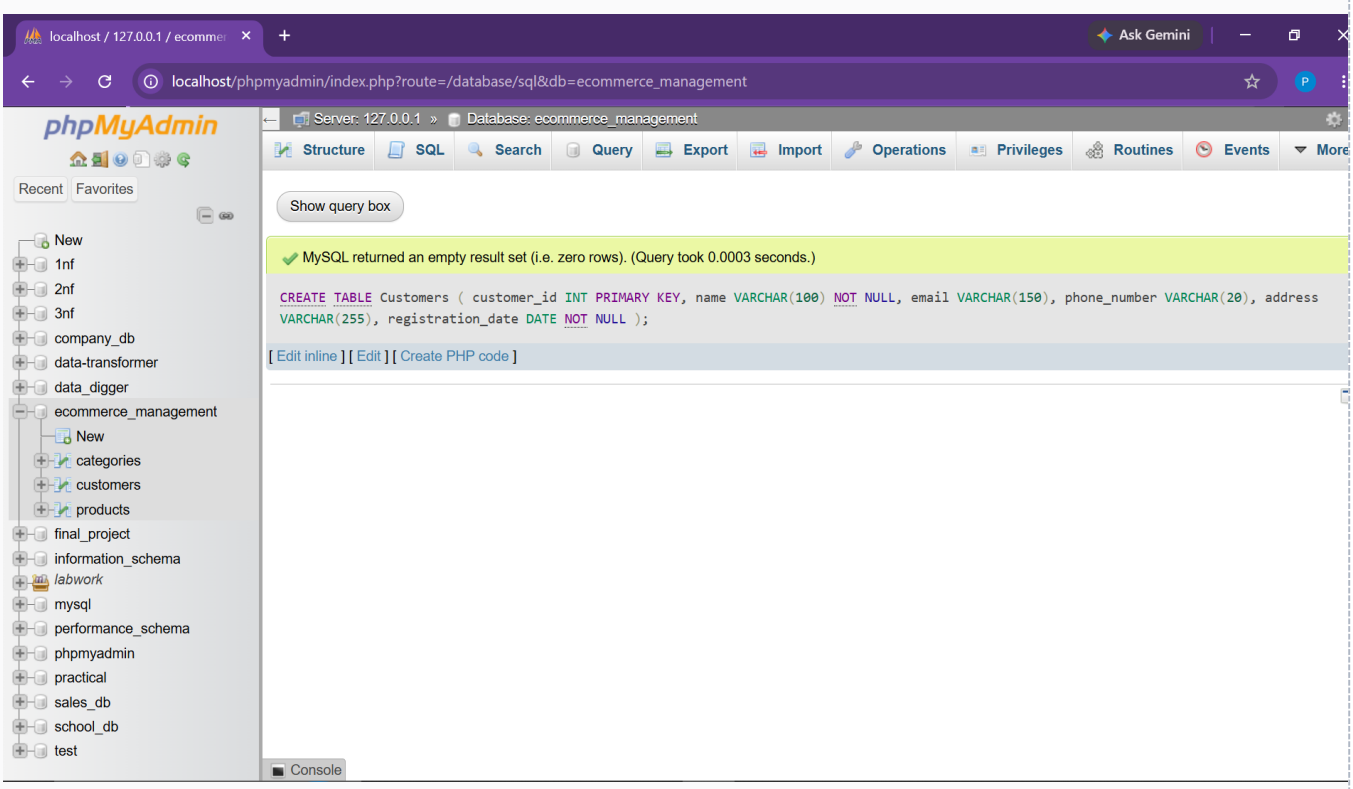


Observation: Products table created with category_id referencing Categories(category_id).

- ### 1.3 Create Customers Table

Objective: Store customer master information including contact details and registration date.

```
CREATE TABLE Customers (
    customer_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150),
    phone_number VARCHAR(20),
    address VARCHAR(255),
    registration_date DATE NOT NULL
);
```

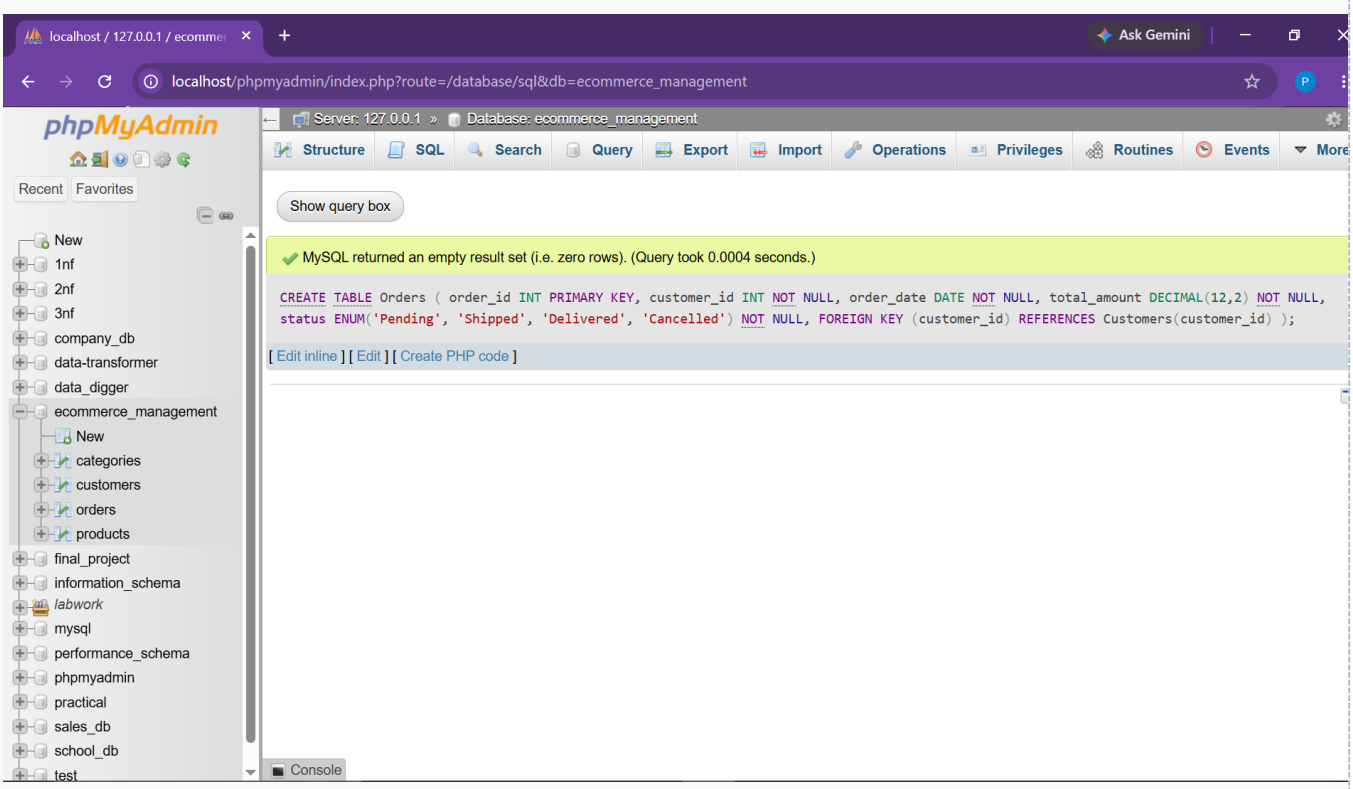


Observation: Customers master entity created with unique customer_id identifier.

- **1.4 Create Orders Table with Foreign Key**

Objective: Manage transaction orders referencing Customer entity with order status ENUM.

```
CREATE TABLE Orders (
    order_id INT PRIMARY KEY,
    customer_id INT NOT NULL,
    order_date DATE NOT NULL,
    total_amount DECIMAL(12,2) NOT NULL,
    status ENUM('Pending', 'Shipped', 'Delivered', 'Cancelled') NOT NULL,
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
);
```

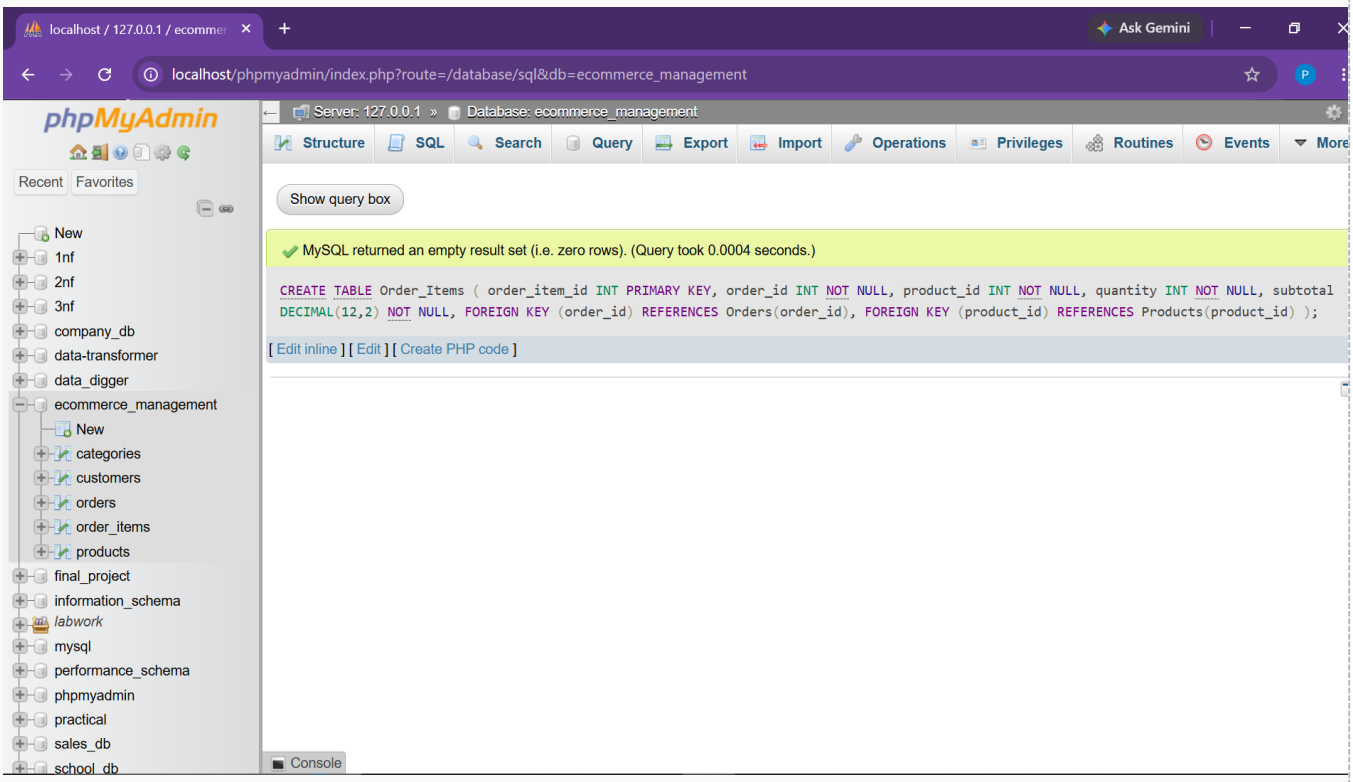


Observation: Orders table created with foreign key linkage to Customers table.

- **1.5 Create Order_Items Table (Junction / Line Items)**

Objective: Link individual order records to respective products with quantity and line subtotals.

```
CREATE TABLE Order_Items (
    order_item_id INT PRIMARY KEY,
    order_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT NOT NULL,
    subtotal DECIMAL(12,2) NOT NULL,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);
```

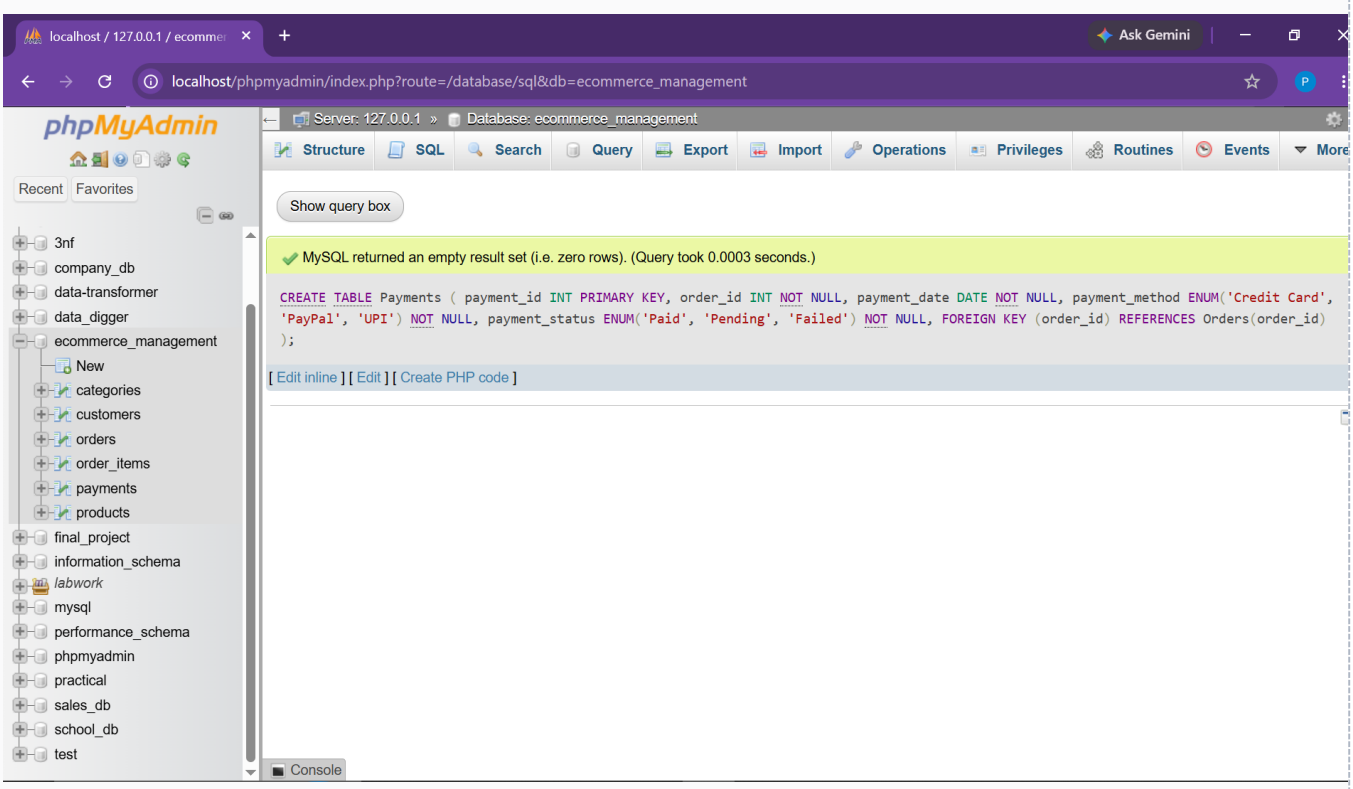


Observation: Order_Items table established with dual foreign key relationships to Orders and Products.

- **1.6 Create Payments Table**

Objective: Track financial settlements, payment methods (Credit Card, PayPal, UPI), and transaction states.

```
CREATE TABLE Payments (
    payment_id INT PRIMARY KEY,
    order_id INT NOT NULL,
    payment_date DATE NOT NULL,
    payment_method ENUM('Credit Card', 'PayPal', 'UPI') NOT NULL,
    payment_status ENUM('Paid', 'Pending', 'Failed') NOT NULL,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id)
);
```

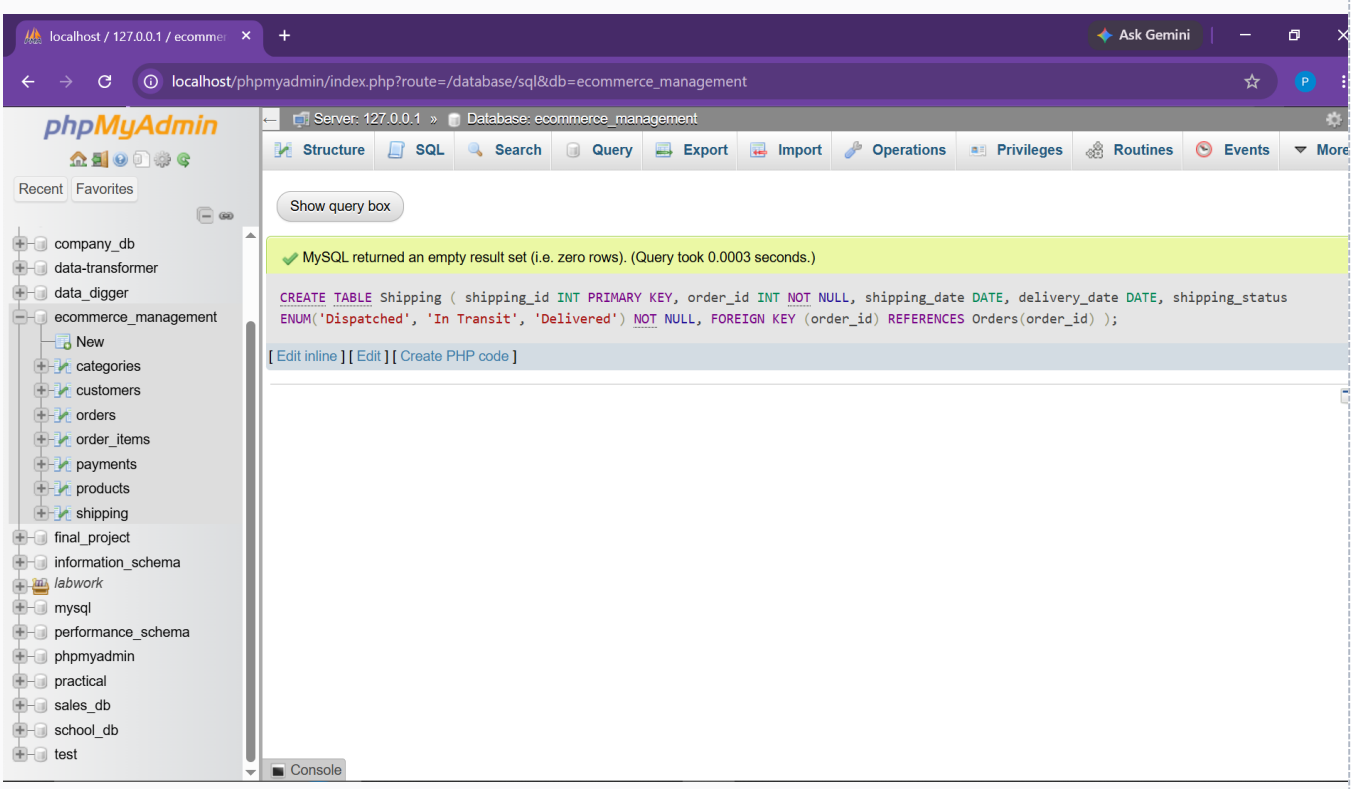


Observation: Payments ledger successfully mapped to Orders table.

- **1.7 Create Shipping Table**

Objective: Track shipment lifecycles, logistics dispatch and delivery dates, and status ENUMs.

```
CREATE TABLE Shipping (
    shipping_id INT PRIMARY KEY,
    order_id INT NOT NULL,
    shipping_date DATE,
    delivery_date DATE,
    shipping_status ENUM('Dispatched', 'In Transit', 'Delivered') NOT NULL,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id)
);
```



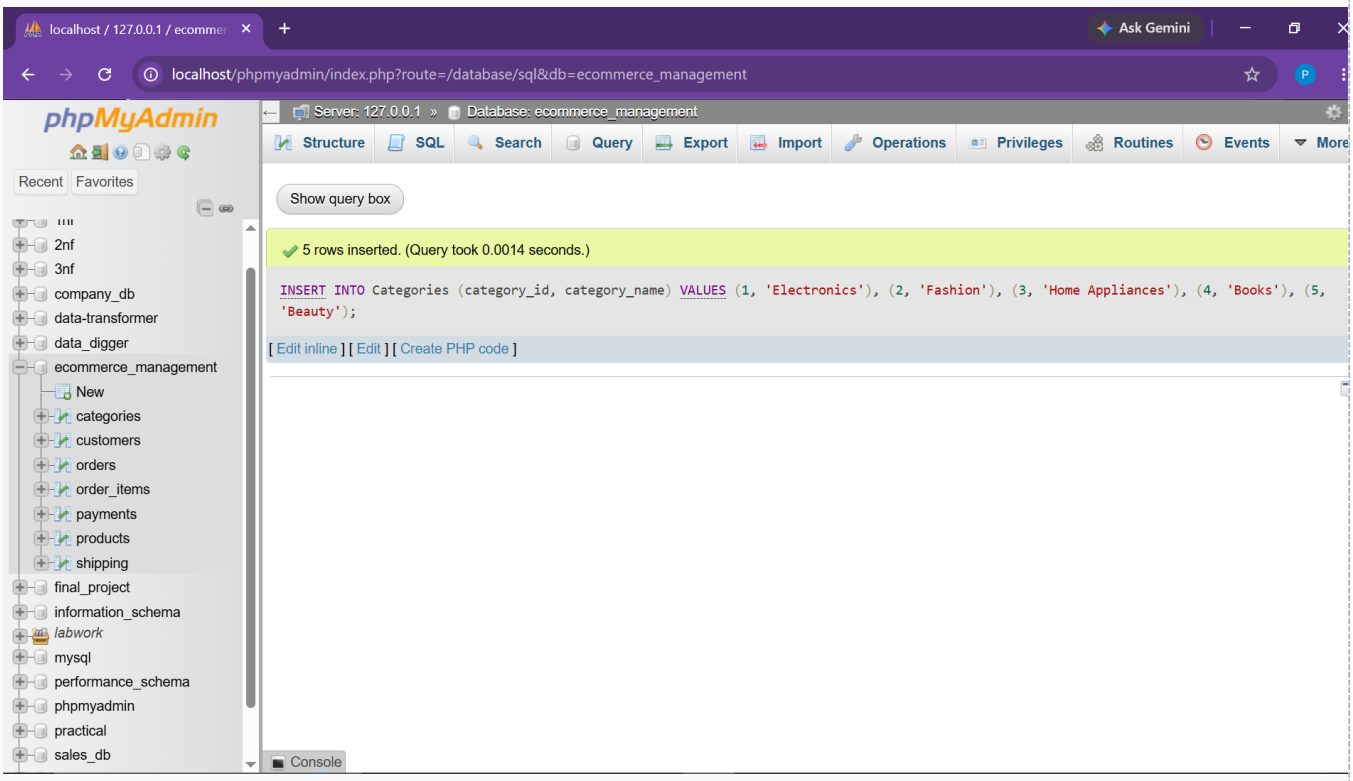
Observation: Shipping entity created with order_id foreign key constraint.

2. SAMPLE DATA SEEDING (DML)

- ### 2.1 Insert Records into Categories

Objective: Seed initial product categories across electronics, fashion, appliances, books, and beauty.

```
INSERT INTO Categories (category_id, category_name) VALUES
(1, 'Electronics'),
(2, 'Fashion'),
(3, 'Home Appliances'),
(4, 'Books'),
(5, 'Beauty');
```

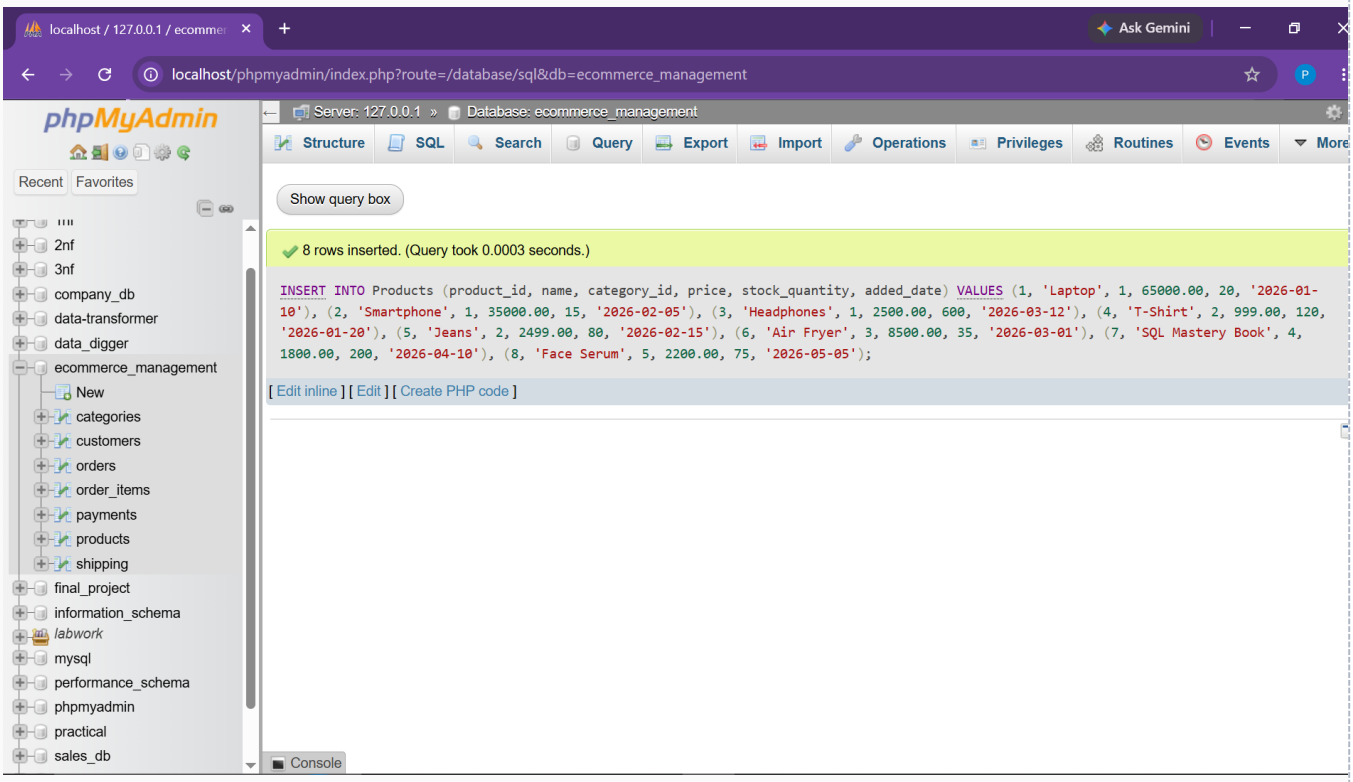


Observation: 5 distinct category rows inserted successfully.

• 2.2 Insert Records into Products

Objective: Populate inventory with products, unit pricing, stock levels, and arrival dates.

```
INSERT INTO Products (product_id, name, category_id, price, stock_quantity, added_date) VALUES
(1, 'Laptop', 1, 65000.00, 20, '2026-01-10'),
(2, 'Smartphone', 1, 35000.00, 15, '2026-02-05'),
(3, 'Headphones', 1, 2500.00, 600, '2026-03-12'),
(4, 'T-Shirt', 2, 999.00, 120, '2026-01-20'),
(5, 'Jeans', 2, 2499.00, 80, '2026-02-15'),
(6, 'Air Fryer', 3, 8500.00, 35, '2026-03-01'),
(7, 'SQL Mastery Book', 4, 1800.00, 200, '2026-04-10'),
(8, 'Face Serum', 5, 2200.00, 75, '2026-05-05');
```

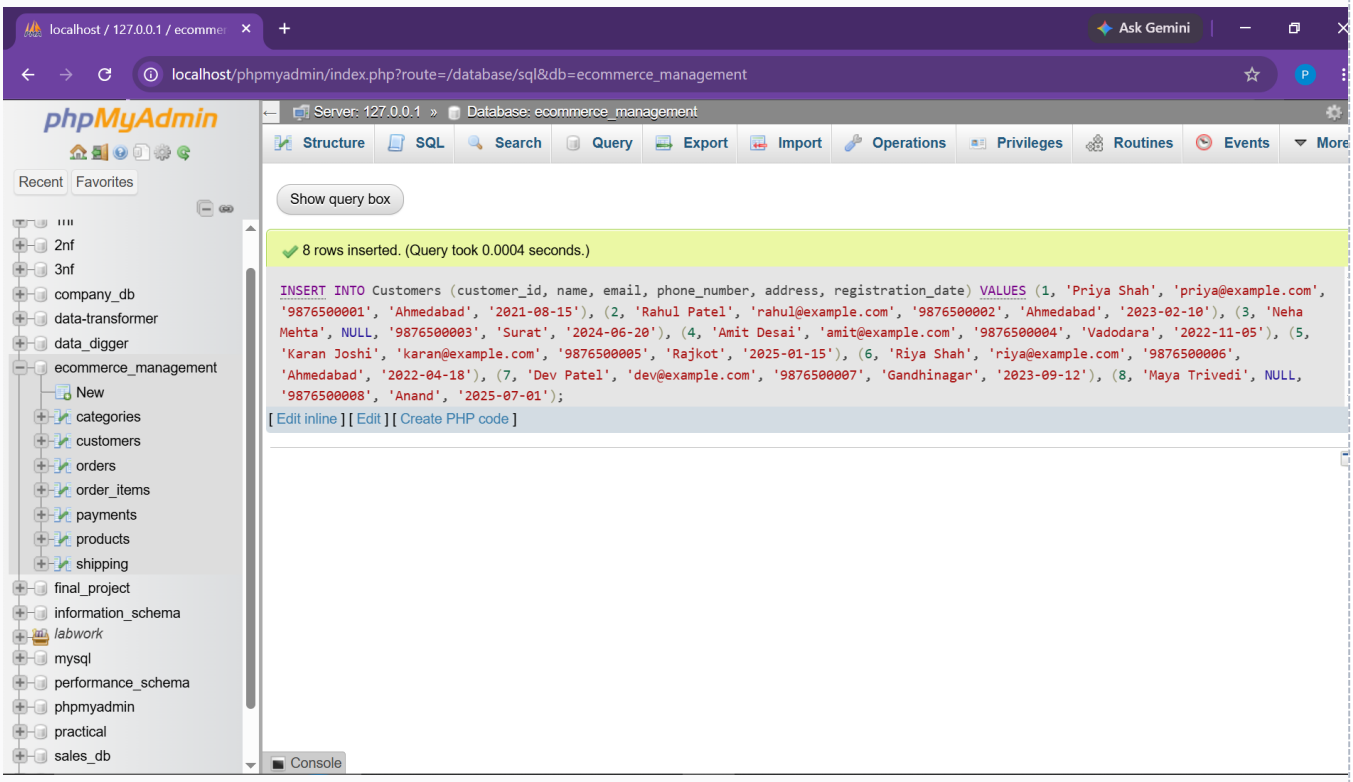


Observation: 8 product catalog records inserted across various merchandise tiers.

• 2.3 Insert Records into Customers

Objective: Insert registered customers across Gujarat cities (Ahmedabad, Surat, Vadodara, Rajkot, etc.).

```
INSERT INTO Customers (customer_id, name, email, phone_number, address, registration_date) VALUES
(1, 'Priya Shah', 'priya@example.com', '9876500001', 'Ahmedabad', '2021-08-15'),
(2, 'Rahul Patel', 'rahul@example.com', '9876500002', 'Ahmedabad', '2023-02-10'),
(3, 'Neha Mehta', NULL, '9876500003', 'Surat', '2024-06-20'),
(4, 'Amit Desai', 'amit@example.com', '9876500004', 'Vadodara', '2022-11-05'),
(5, 'Karan Joshi', 'karan@example.com', '9876500005', 'Rajkot', '2025-01-15'),
(6, 'Riya Shah', 'riya@example.com', '9876500006', 'Ahmedabad', '2022-04-18'),
(7, 'Dev Patel', 'dev@example.com', '9876500007', 'Gandhinagar', '2023-09-12'),
(8, 'Maya Trivedi', NULL, '9876500008', 'Anand', '2025-07-01');
```

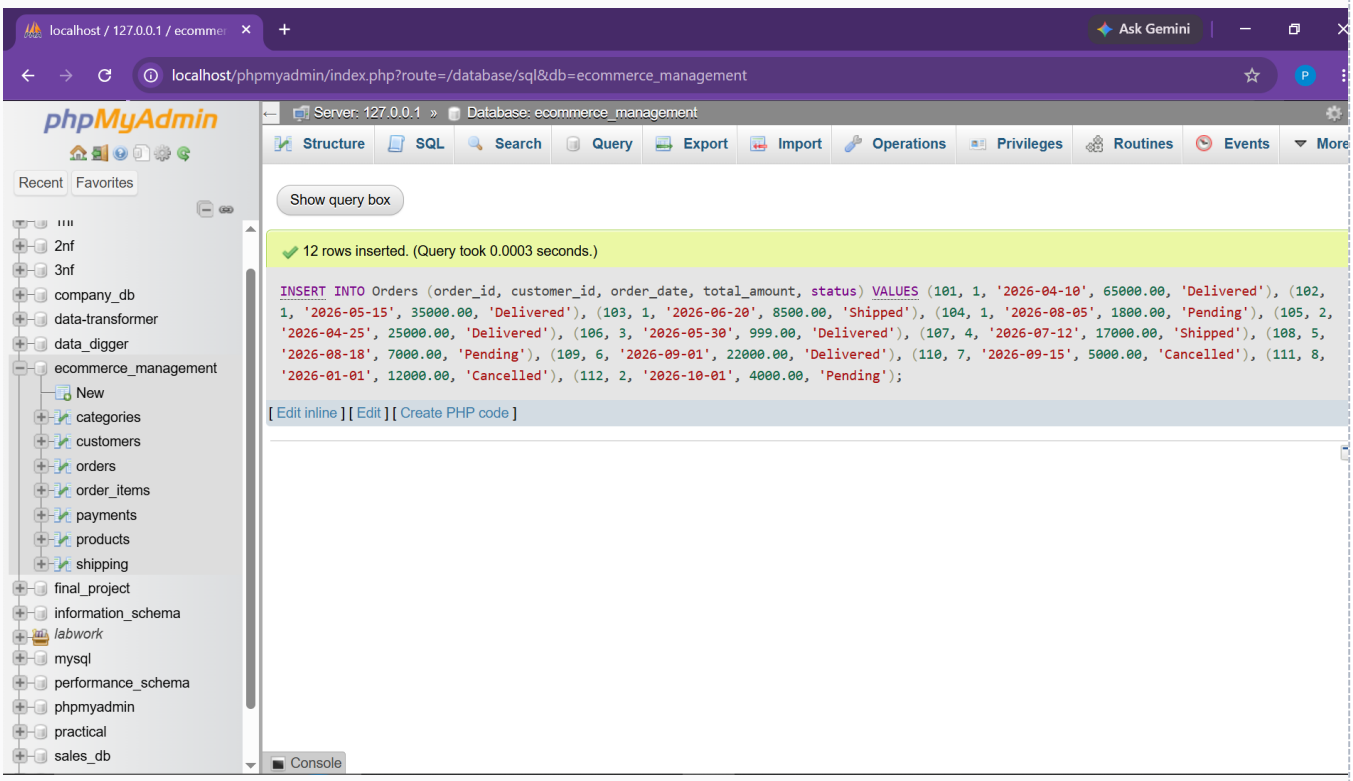


Observation: 8 customer profiles seeded, including test entries with NULL email addresses.

- **2.4 Insert Records into Orders**

Objective: Seed order transactions with various fulfillment statuses.

```
INSERT INTO Orders (order_id, customer_id, order_date, total_amount, status) VALUES
(101, 1, '2026-04-10', 65000.00, 'Delivered'),
(102, 1, '2026-05-15', 35000.00, 'Delivered'),
(103, 1, '2026-06-20', 8500.00, 'Shipped'),
(104, 1, '2026-08-05', 1800.00, 'Pending'),
(105, 2, '2026-04-25', 25000.00, 'Delivered'),
(106, 3, '2026-05-30', 999.00, 'Delivered'),
(107, 4, '2026-07-12', 17000.00, 'Shipped'),
(108, 5, '2026-08-18', 7000.00, 'Pending'),
(109, 6, '2026-09-01', 22000.00, 'Delivered'),
(110, 7, '2026-09-15', 5000.00, 'Cancelled'),
(111, 8, '2026-01-01', 12000.00, 'Cancelled'),
(112, 2, '2026-10-01', 4000.00, 'Pending');
```

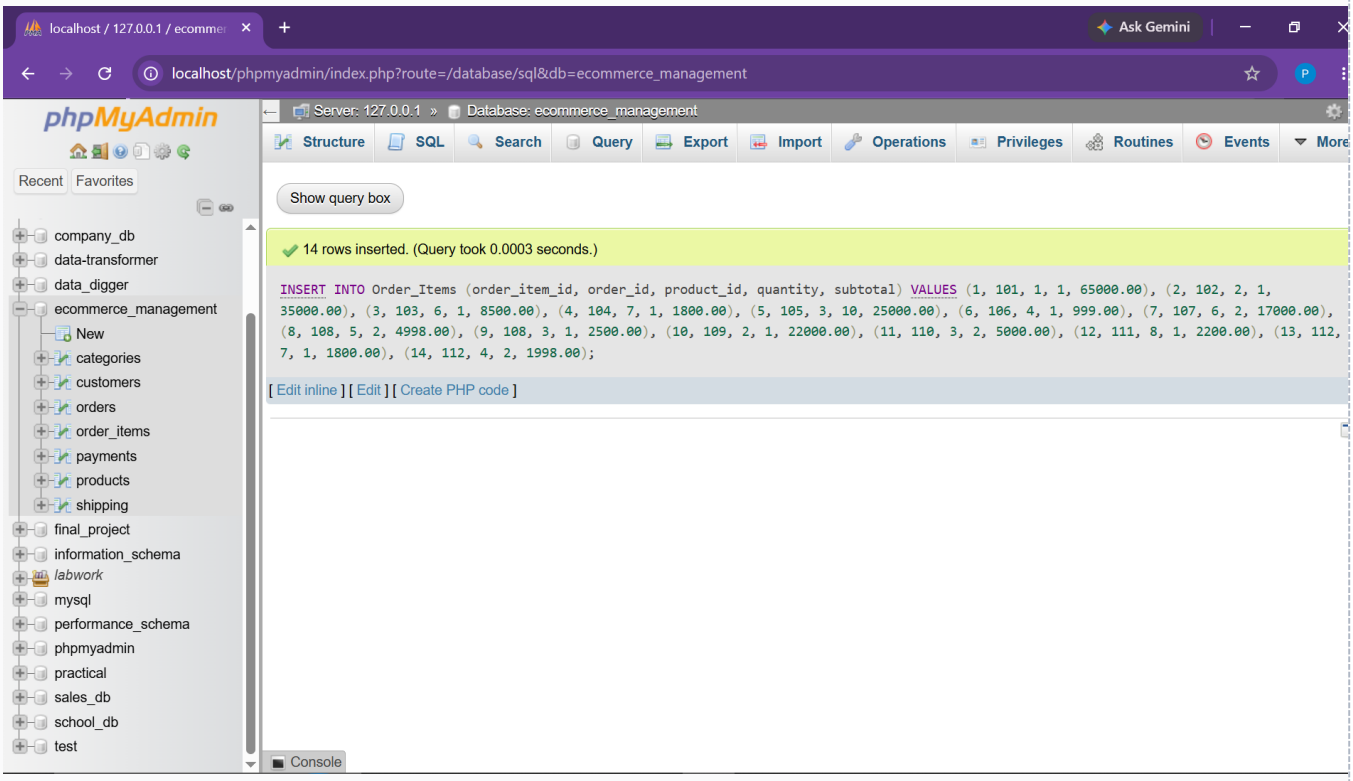


Observation: 12 order records successfully added with Delivered, Shipped, Pending, and Cancelled statuses.

- **2.5 Insert Records into Order_Items**

Objective: Map order line items and calculated product subtotals.

```
INSERT INTO Order_Items (order_item_id, order_id, product_id, quantity, subtotal) VALUES
(1, 101, 1, 1, 65000.00), (2, 102, 2, 1, 35000.00), (3, 103, 6, 1, 8500.00),
(4, 104, 7, 1, 1800.00),  (5, 105, 3, 10, 25000.00), (6, 106, 4, 1, 999.00),
(7, 107, 6, 2, 17000.00), (8, 108, 5, 2, 4998.00),  (9, 108, 3, 1, 2500.00),
(10, 109, 2, 1, 22000.00),(11, 110, 3, 2, 5000.00), (12, 111, 8, 1, 2200.00),
(13, 112, 7, 1, 1800.00), (14, 112, 4, 2, 1998.00);
```

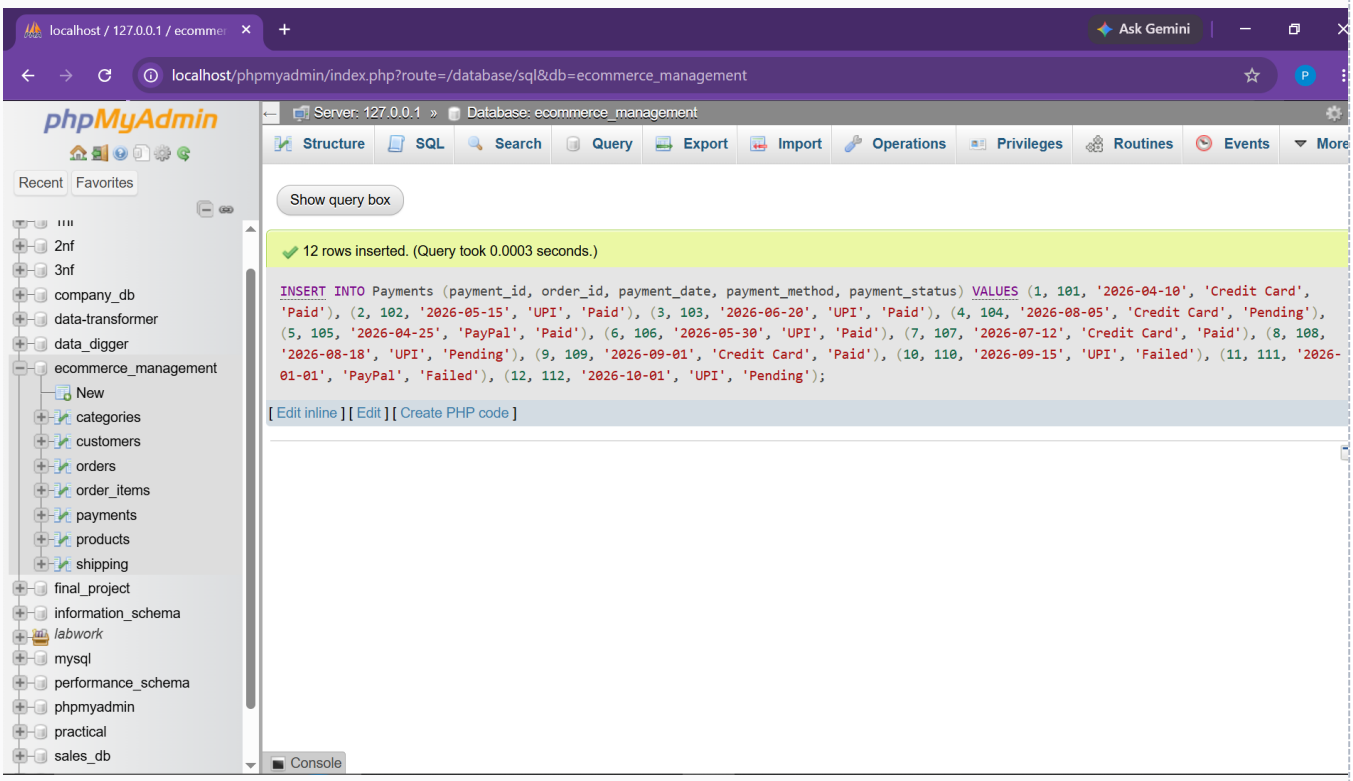


Observation: 14 individual order item rows seeded to preserve product unit granularity.

- **2.6 Insert Records into Payments**

Objective: Seed financial settlements across Credit Card, UPI, and PayPal gateways.

```
INSERT INTO Payments (payment_id, order_id, payment_date, payment_method, payment_status) VALUES
(1, 101, '2026-04-10', 'Credit Card', 'Paid'),
(2, 102, '2026-05-15', 'UPI', 'Paid'),
(3, 103, '2026-06-20', 'UPI', 'Paid'),
(4, 104, '2026-08-05', 'Credit Card', 'Pending'),
(5, 105, '2026-04-25', 'PayPal', 'Paid'),
(6, 106, '2026-05-30', 'UPI', 'Paid'),
(7, 107, '2026-07-12', 'Credit Card', 'Paid'),
(8, 108, '2026-08-18', 'UPI', 'Pending'),
(9, 109, '2026-09-01', 'Credit Card', 'Paid'),
(10, 110, '2026-09-15', 'UPI', 'Failed'),
(11, 111, '2026-01-01', 'PayPal', 'Failed'),
(12, 112, '2026-10-01', 'UPI', 'Pending');
```

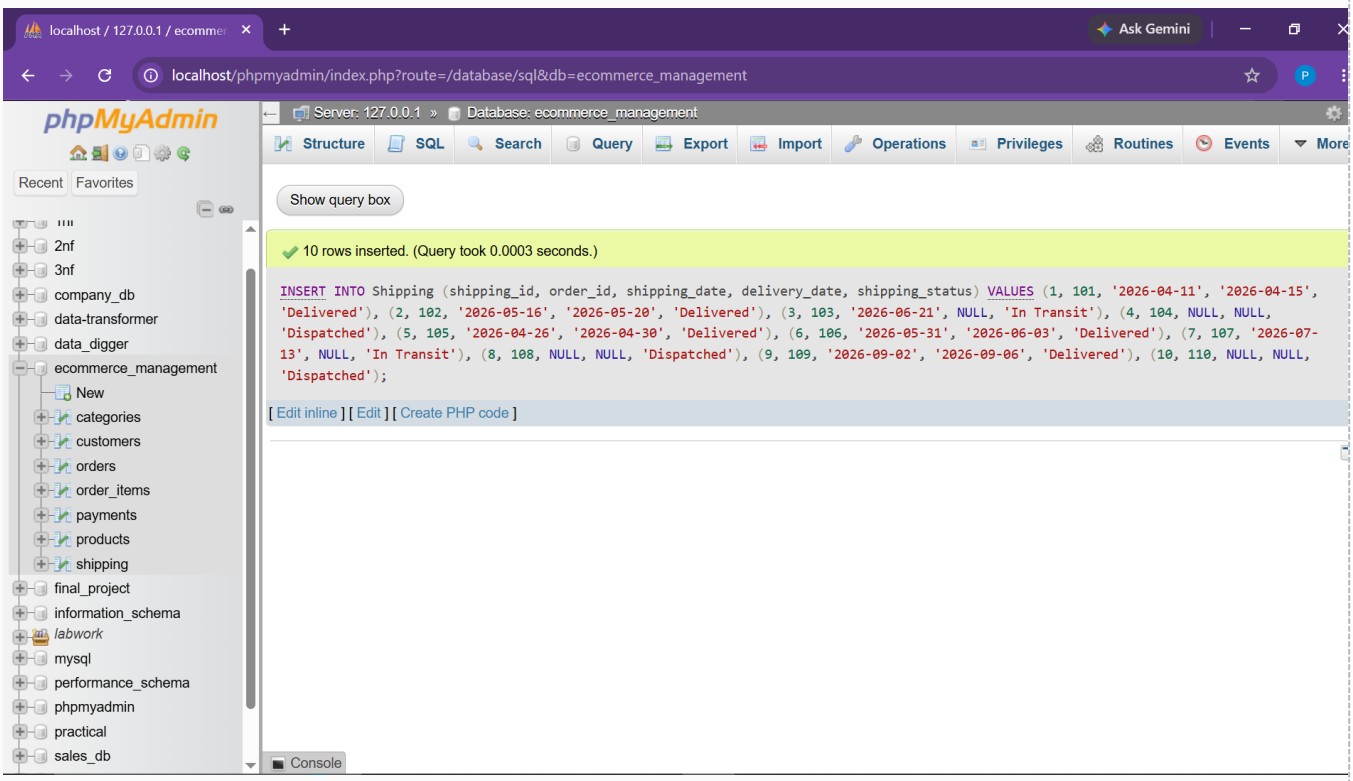


Observation: 12 transaction payment records created reflecting Paid, Pending, and Failed states.

- **2.7 Insert Records into Shipping**

Objective: Seed logistics lifecycle tracking for dispatched, in-transit, and delivered orders.

```
INSERT INTO Shipping (shipping_id, order_id, shipping_date, delivery_date, shipping_status) VALUES
(1, 101, '2026-04-11', '2026-04-15', 'Delivered'),
(2, 102, '2026-05-16', '2026-05-20', 'Delivered'),
(3, 103, '2026-06-21', NULL, 'In Transit'),
(4, 104, NULL, NULL, 'Dispatched'),
(5, 105, '2026-04-26', '2026-04-30', 'Delivered'),
(6, 106, '2026-05-31', '2026-06-03', 'Delivered'),
(7, 107, '2026-07-13', NULL, 'In Transit'),
(8, 108, NULL, NULL, 'Dispatched'),
(9, 109, '2026-09-02', '2026-09-06', 'Delivered'),
(10, 110, NULL, NULL, 'Dispatched');
```



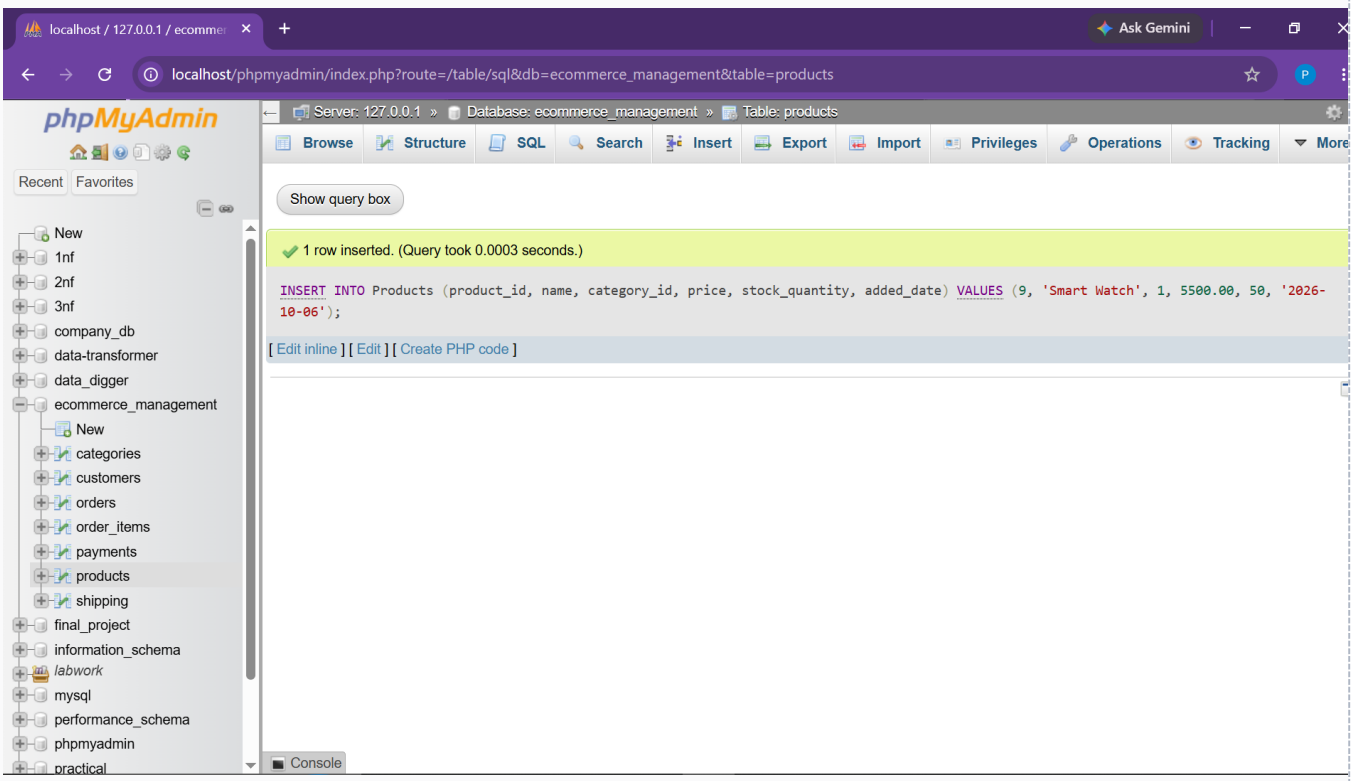
Observation: 10 shipment records logged with delivery timelines and logistics status.

3. PRACTICAL TASKS & ANALYTICAL EXECUTION

- **Task 1.1 Insert New Product (CRUD: Insert)**

Objective: Add a new product record to the Products catalog.

```
INSERT INTO Products (product_id, name, category_id, price, stock_quantity, added_date)
VALUES (9, 'Smart Watch', 1, 5500.00, 50, '2026-10-06');
```

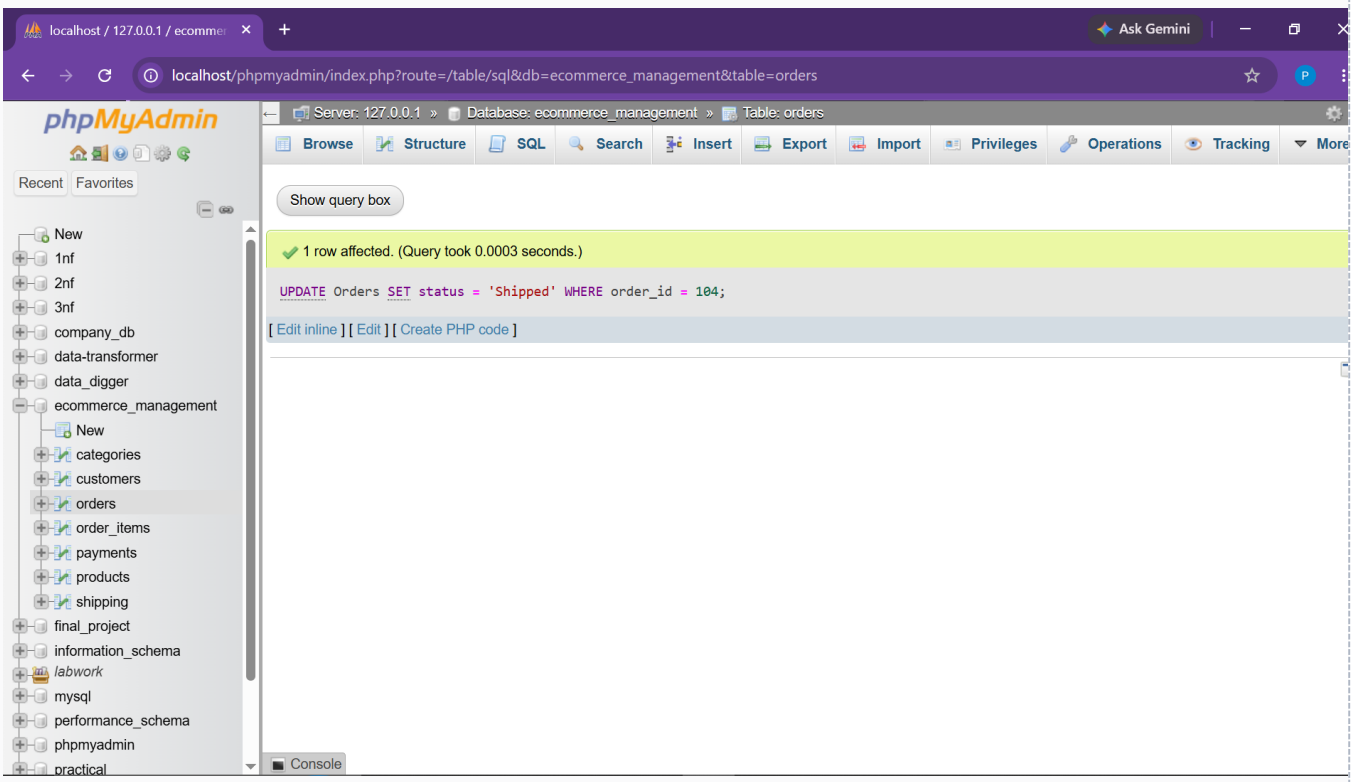


Observation: Successfully inserted 'Smart Watch' (product_id: 9) into the database.

- **Task 1.2 Update Stock / Order Status (CRUD: Update)**

Objective: Update the status of an existing order when fulfillment occurs.

```
UPDATE Orders
SET status = 'Shipped'
WHERE order_id = 104;
```

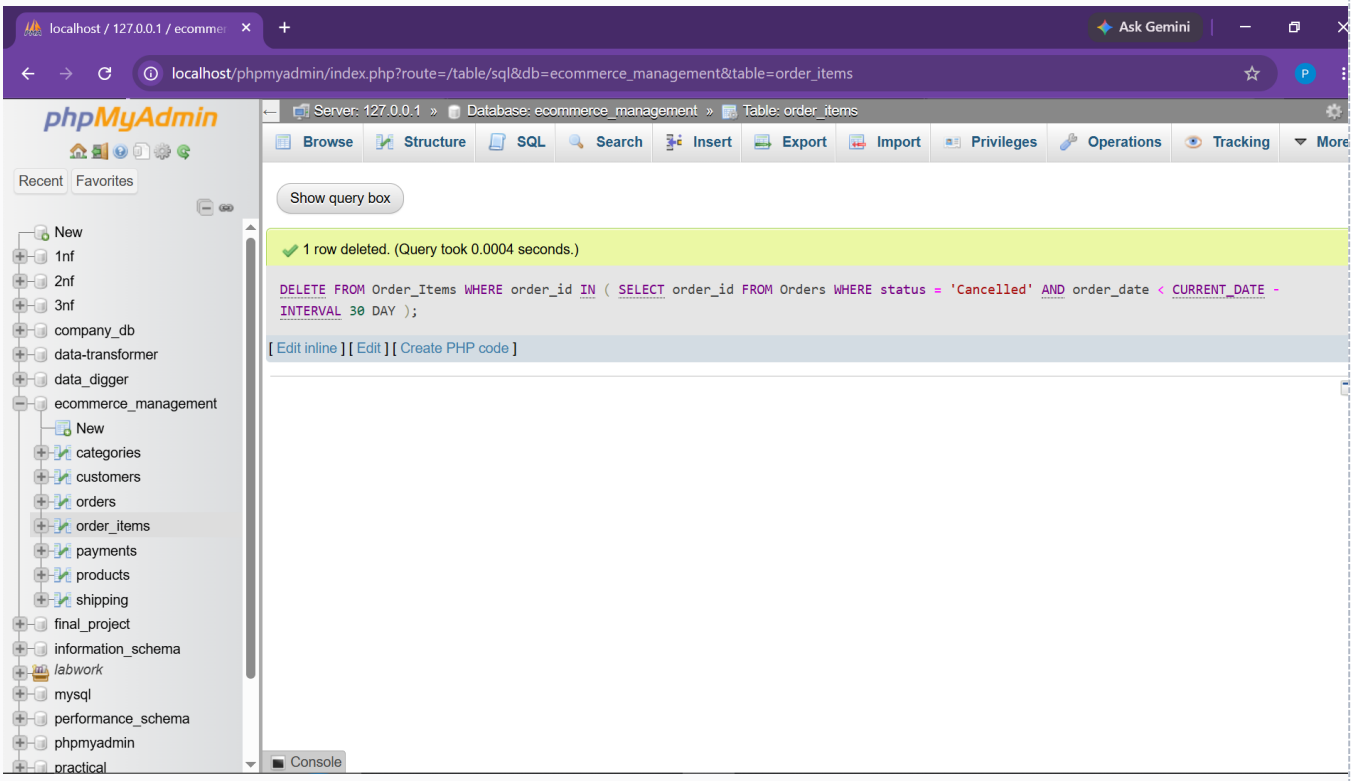


Observation: 1 row updated. Order 104 status transitioned from 'Pending' to 'Shipped'.

- **Task 1.3 Delete Cancelled Orders Older Than 30 Days (CRUD: Delete)**

Objective: Purge old cancelled order item dependencies based on order date interval.

```
DELETE FROM Order_Items
WHERE order_id IN (
    SELECT order_id
    FROM Orders
    WHERE status = 'Cancelled'
    AND order_date < CURRENT_DATE - INTERVAL 30 DAY
);
```

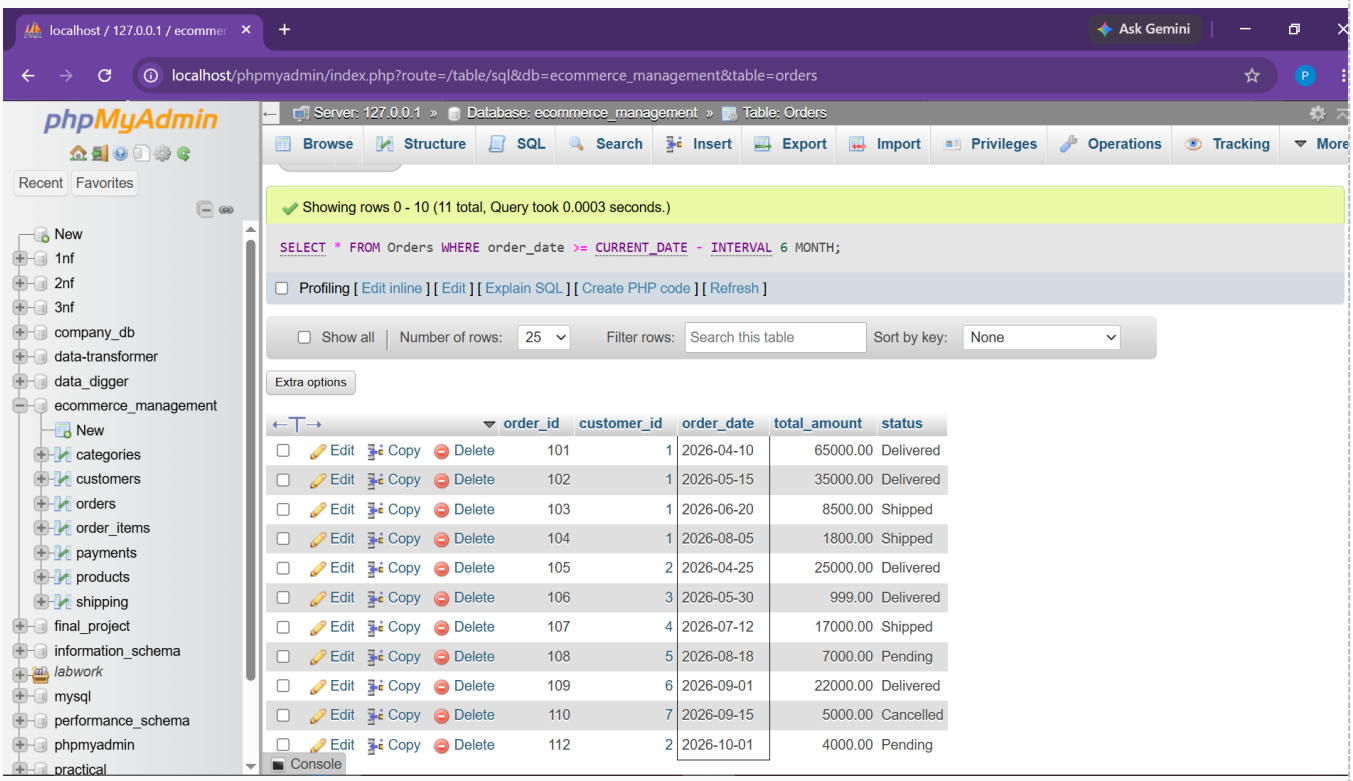


Observation: Rows matching historical cancelled orders purged successfully without violating relational constraints.

- **Task 2.1 Find All Orders Placed in Last 6 Months (WHERE)**

Objective: Filter orders occurring within the recent 6 month rolling window.

```
SELECT *
FROM Orders
WHERE order_date >= CURRENT_DATE - INTERVAL 6 MONTH;
```

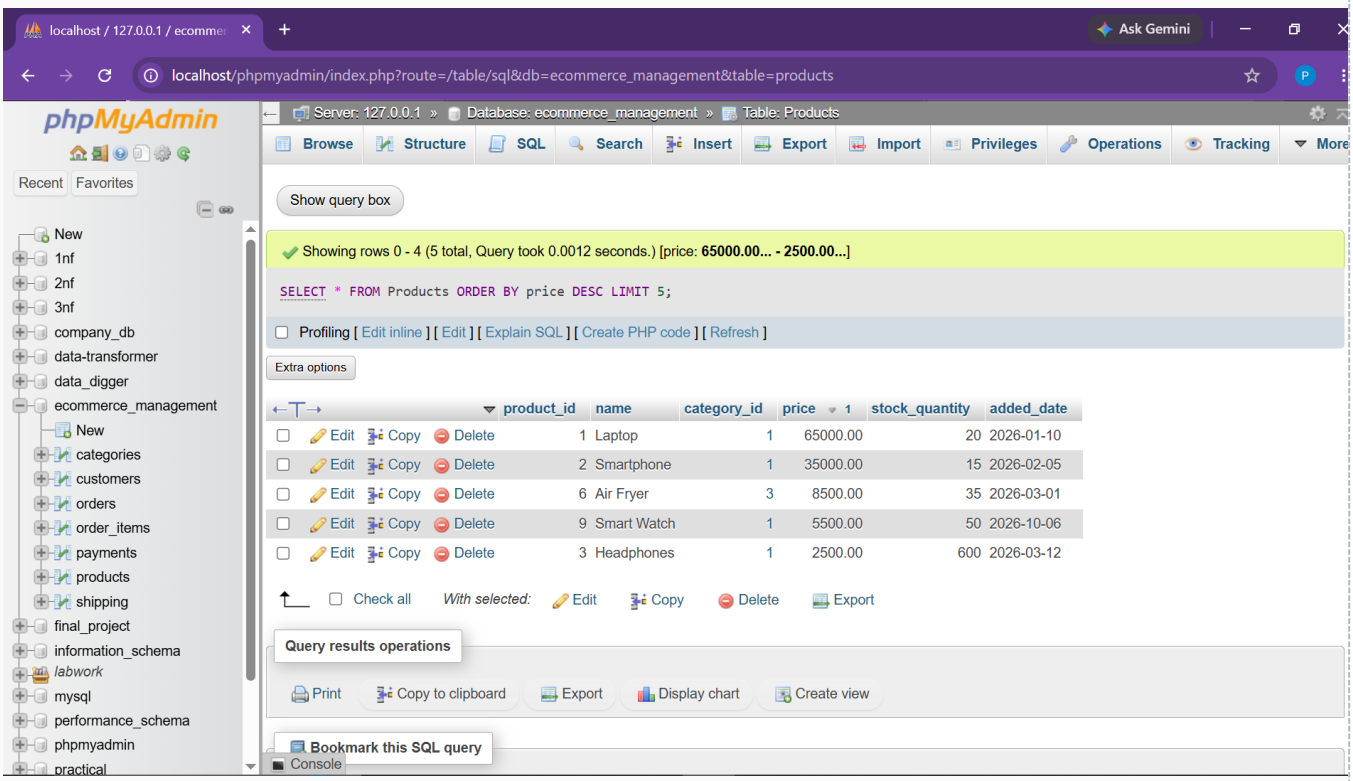


Observation: Returned all orders placed in the last 6 months based on dynamic CURRENT_DATE calculations.

- **Task 2.2 Get Top 5 Highest-Priced Products (ORDER BY & LIMIT)**

Objective: Sort catalog items by price in descending order and retrieve the top 5.

```
SELECT *
FROM Products
ORDER BY price DESC
LIMIT 5;
```

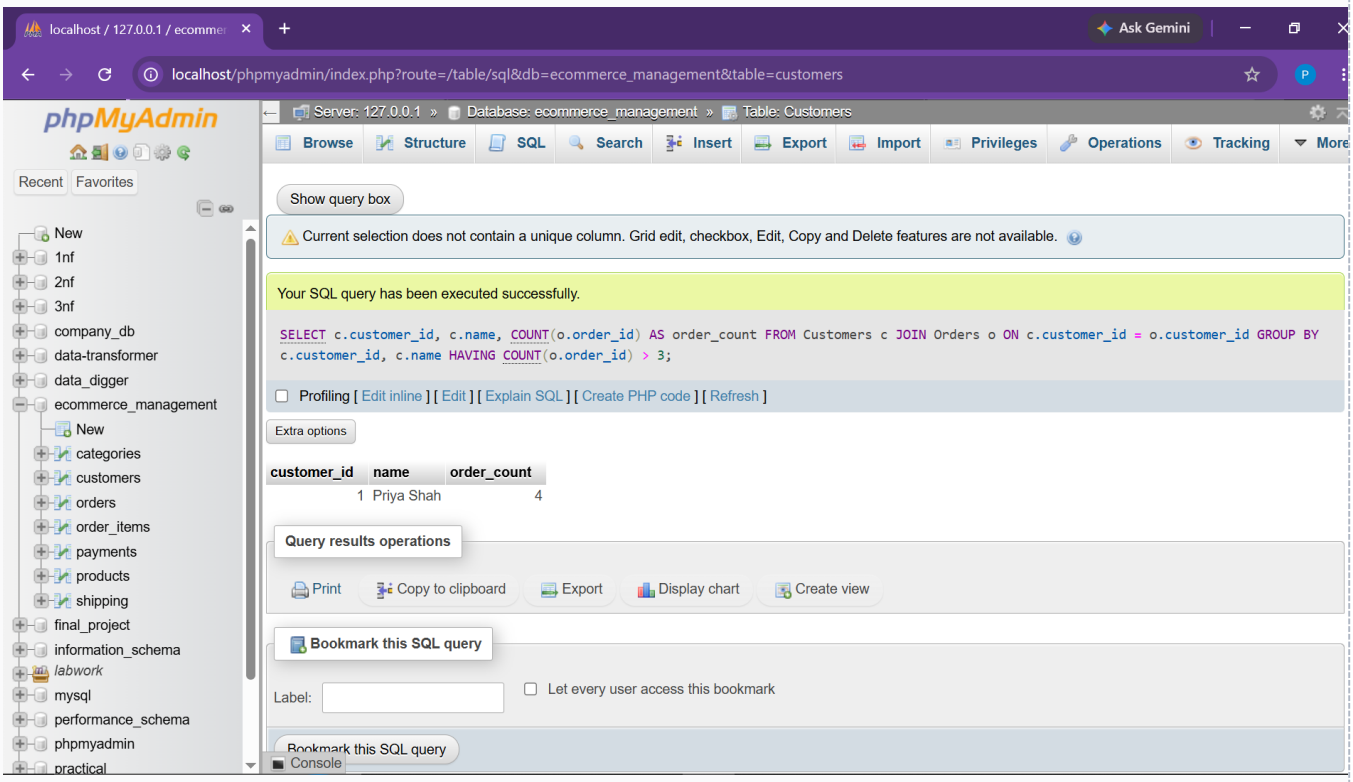


Observation: Returned 5 highest value products led by Laptop (₹65,000.00) and Smartphone (₹35,000.00).

- **Task 2.3 Customers Who Placed More Than 3 Orders (HAVING)**

Objective: Aggregate order counts per customer and filter those with high transaction volume.

```
SELECT
    c.customer_id,
    c.name,
    COUNT(o.order_id) AS order_count
FROM Customers c
JOIN Orders o
    ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name
HAVING COUNT(o.order_id) > 3;
```

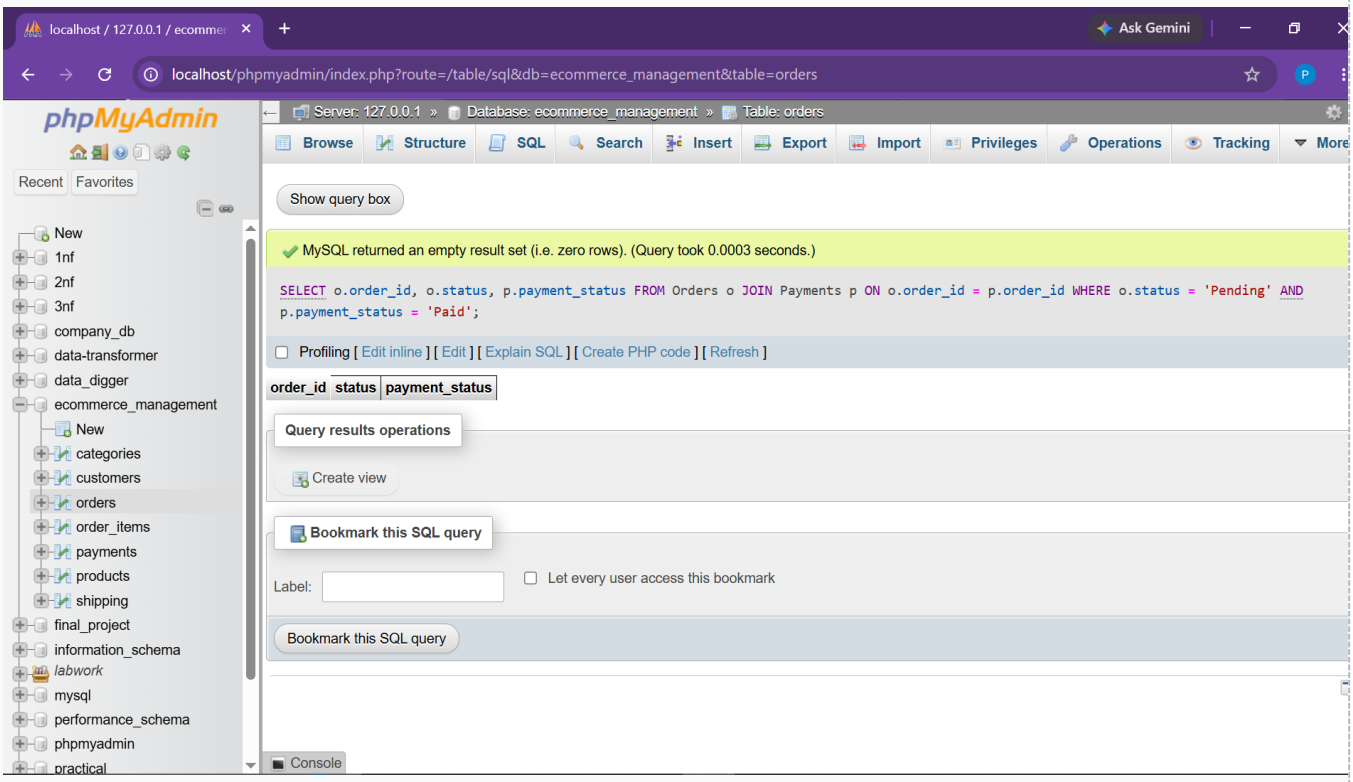


Observation: Successfully identified Priya Shah (customer_id: 1) with 4 total orders.

- **Task 3.1 Orders Where Status = 'Pending' AND Payment = 'Paid' (AND)**

Objective: Identify pending fulfillment orders whose payment has already been verified.

```
SELECT
    o.order_id,
    o.status,
    p.payment_status
FROM Orders o
JOIN Payments p
    ON o.order_id = p.order_id
WHERE o.status = 'Pending'
AND p.payment_status = 'Paid';
```

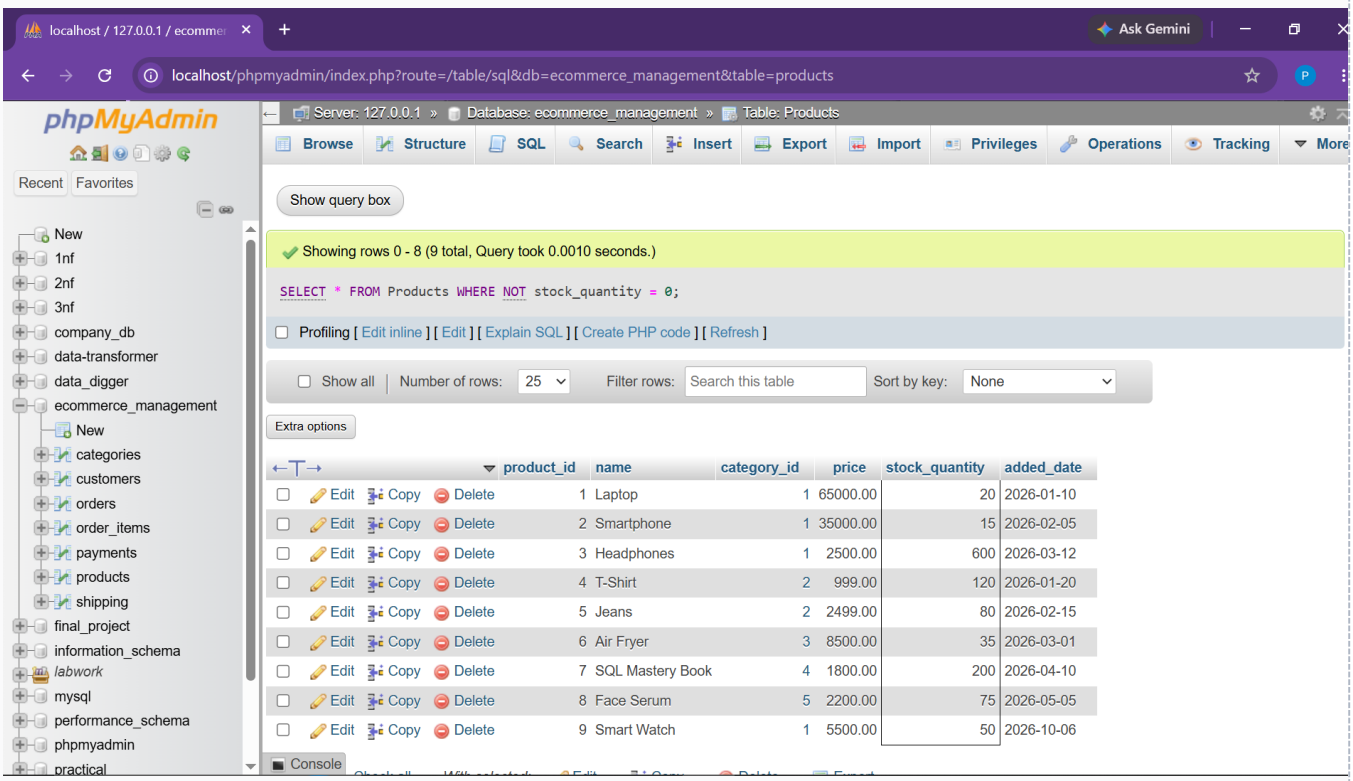


Observation: Evaluated join of Orders and Payments to isolate pending orders with paid status.

- **Task 3.2 Find Products That Are NOT Out of Stock (NOT)**

Objective: List all active products that maintain inventory stock greater than zero.

```
SELECT *
FROM Products
WHERE NOT stock_quantity = 0;
```

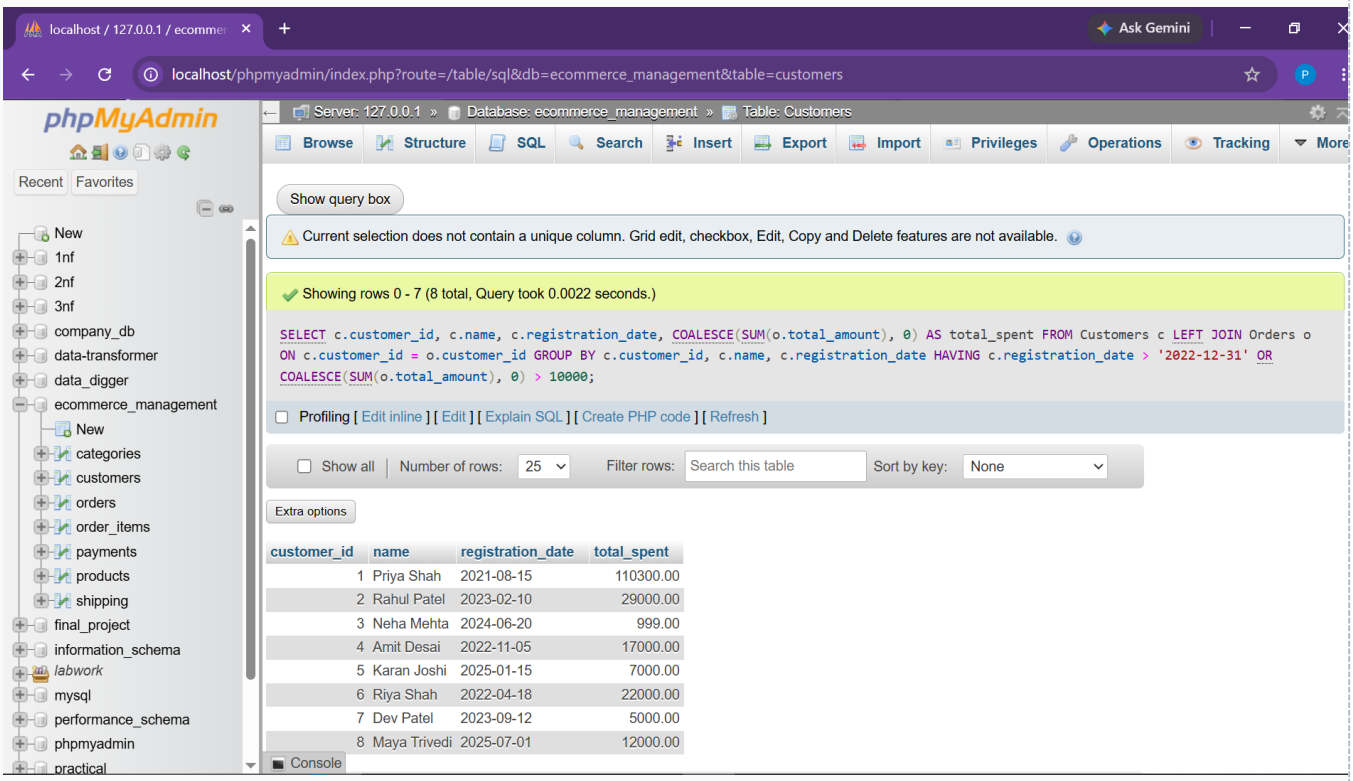


Observation: Returned all active products where stock_quantity is not zero.

- **Task 3.3 Customers Registered After 2022 OR Spent > ₹10,000 (OR / HAVING)**

Objective: Filter customers satisfying either recent acquisition date or high lifetime value.

```
SELECT
    c.customer_id,
    c.name,
    c.registration_date,
    COALESCE(SUM(o.total_amount), 0) AS total_spent
FROM Customers c
LEFT JOIN Orders o
    ON c.customer_id = o.customer_id
GROUP BY
    c.customer_id,
    c.name,
    c.registration_date
HAVING c.registration_date > '2022-12-31'
    OR COALESCE(SUM(o.total_amount), 0) > 10000;
```

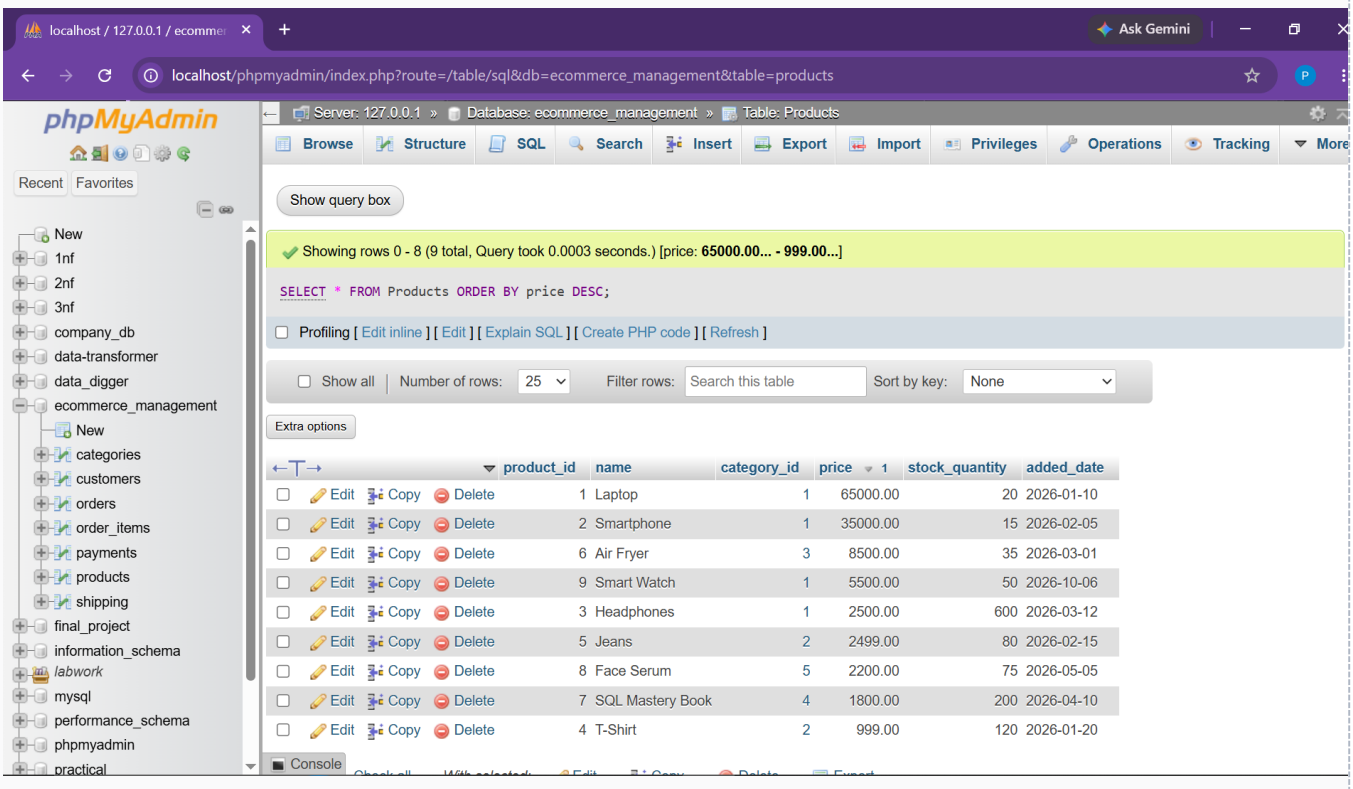


Observation: Correctly evaluated customer cohort based on registration timeframe or aggregate total spent.

- **Task 4.1 Products Sorted by Price in Descending Order (ORDER BY)**

Objective: Sort catalog items from highest price to lowest price.

```
SELECT *
FROM Products
ORDER BY price DESC;
```

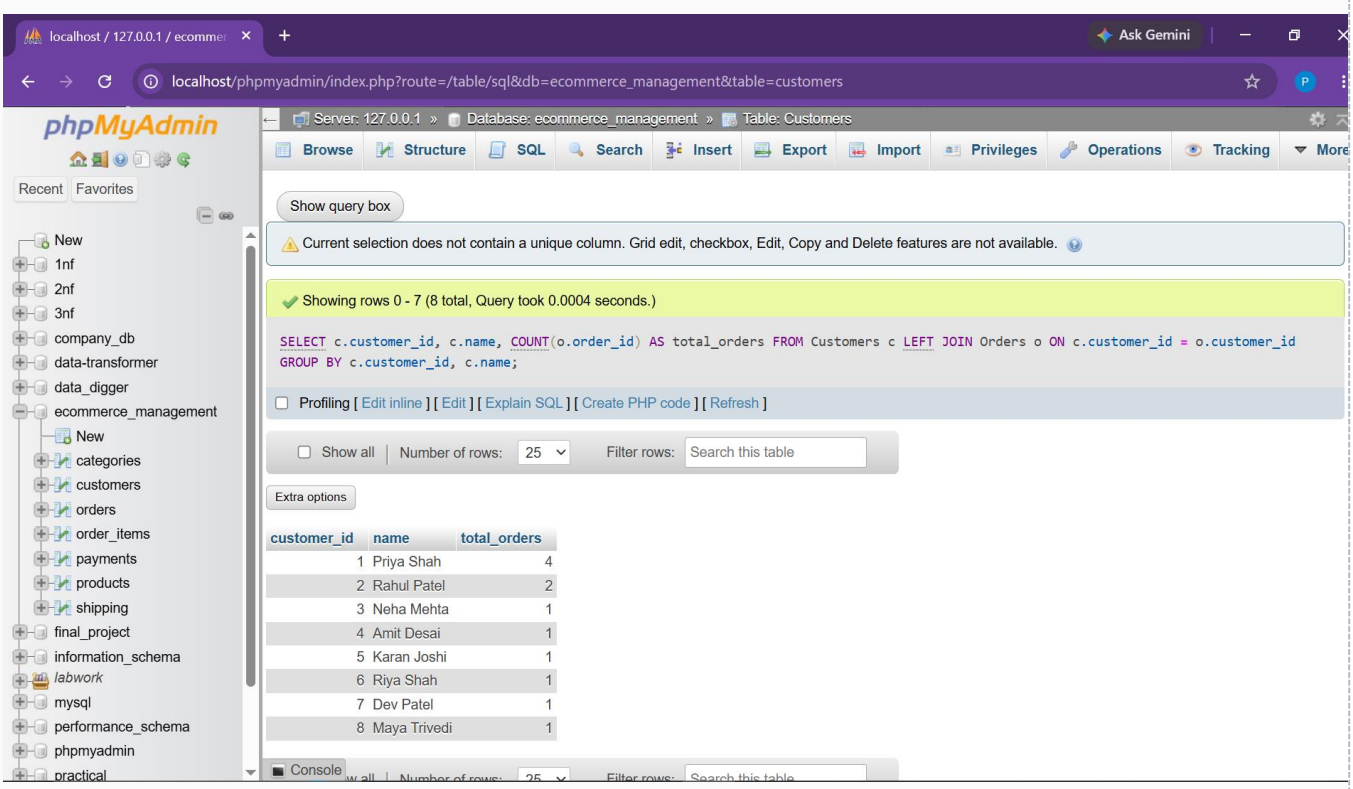


Observation: Catalog ordered sequentially from Laptop (₹65,000.00) down to T-Shirt (₹999.00).

- **Task 4.2 Display Number of Orders Placed by Each Customer (GROUP BY)**

Objective: Group transaction orders by customer entity to summarize purchasing activity.

```
SELECT
    c.customer_id,
    c.name,
    COUNT(o.order_id) AS total_orders
FROM Customers c
LEFT JOIN Orders o
    ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name;
```

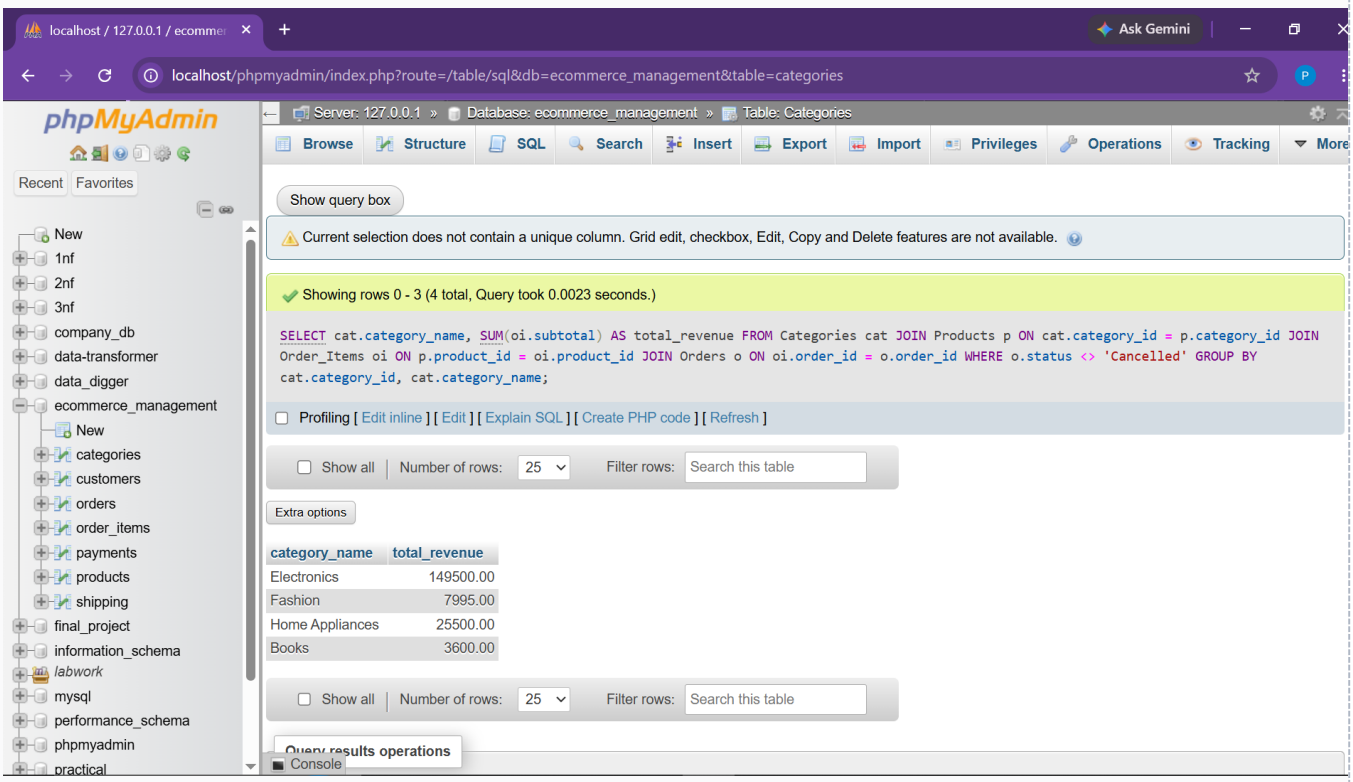


Observation: Returned all 8 registered customers along with their respective order frequencies.

- **Task 4.3 Total Revenue Generated per Category (GROUP BY with Joins)**

Objective: Aggregate revenue grouped by merchandise category excluding cancelled transactions.

```
SELECT
    cat.category_name,
    SUM(oi.subtotal) AS total_revenue
FROM Categories cat
JOIN Products p
    ON cat.category_id = p.category_id
JOIN Order_Items oi
    ON p.product_id = oi.product_id
JOIN Orders o
    ON oi.order_id = o.order_id
WHERE o.status <> 'Cancelled'
GROUP BY cat.category_id, cat.category_name;
```

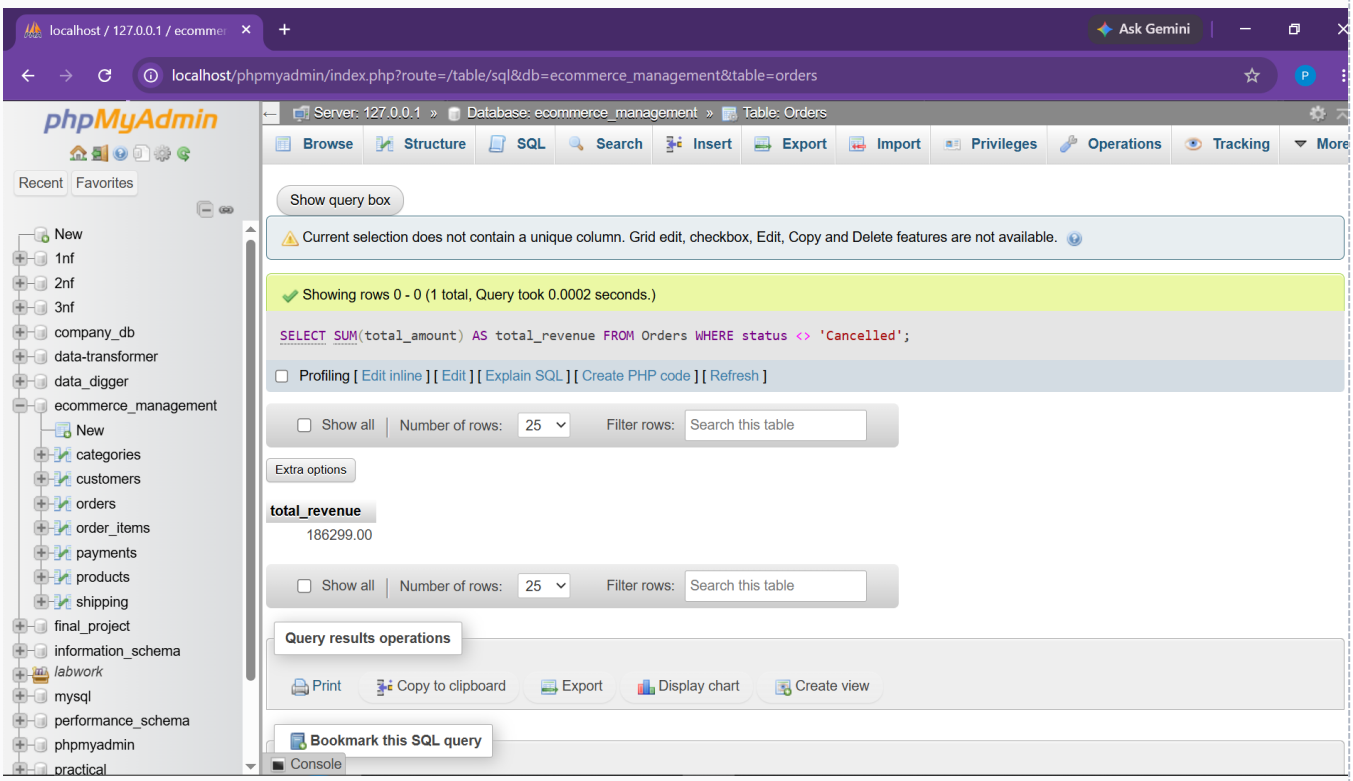


Observation: Summarized total revenue categorized into Electronics, Fashion, Home Appliances, and Books.

- **Task 5.1 Find Total Store Revenue (SUM)**

Objective: Compute cumulative gross revenue generated across all valid completed orders.

```
SELECT SUM(total_amount) AS total_revenue
FROM Orders
WHERE status <> 'Cancelled';
```

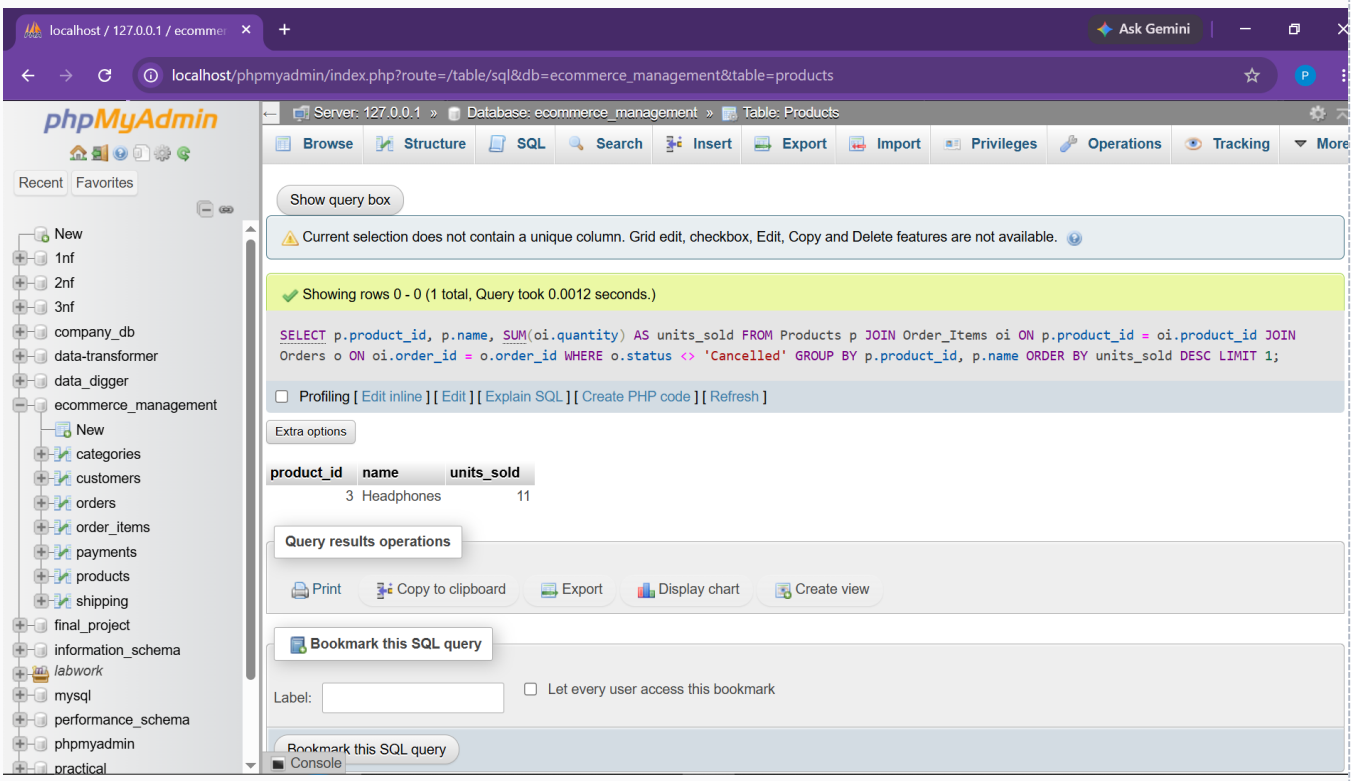


Observation: Returned store revenue after strictly filtering out cancelled orders.

- **Task 5.2 Identify the Most Purchased Product (SUM with LIMIT)**

Objective: Determine product SKU with highest units sold in the order item ledger.

```
SELECT
    p.product_id,
    p.name,
    SUM(oi.quantity) AS units_sold
FROM Products p
JOIN Order_Items oi
    ON p.product_id = oi.product_id
JOIN Orders o
    ON oi.order_id = o.order_id
WHERE o.status <> 'Cancelled'
GROUP BY p.product_id, p.name
ORDER BY units_sold DESC
LIMIT 1;
```

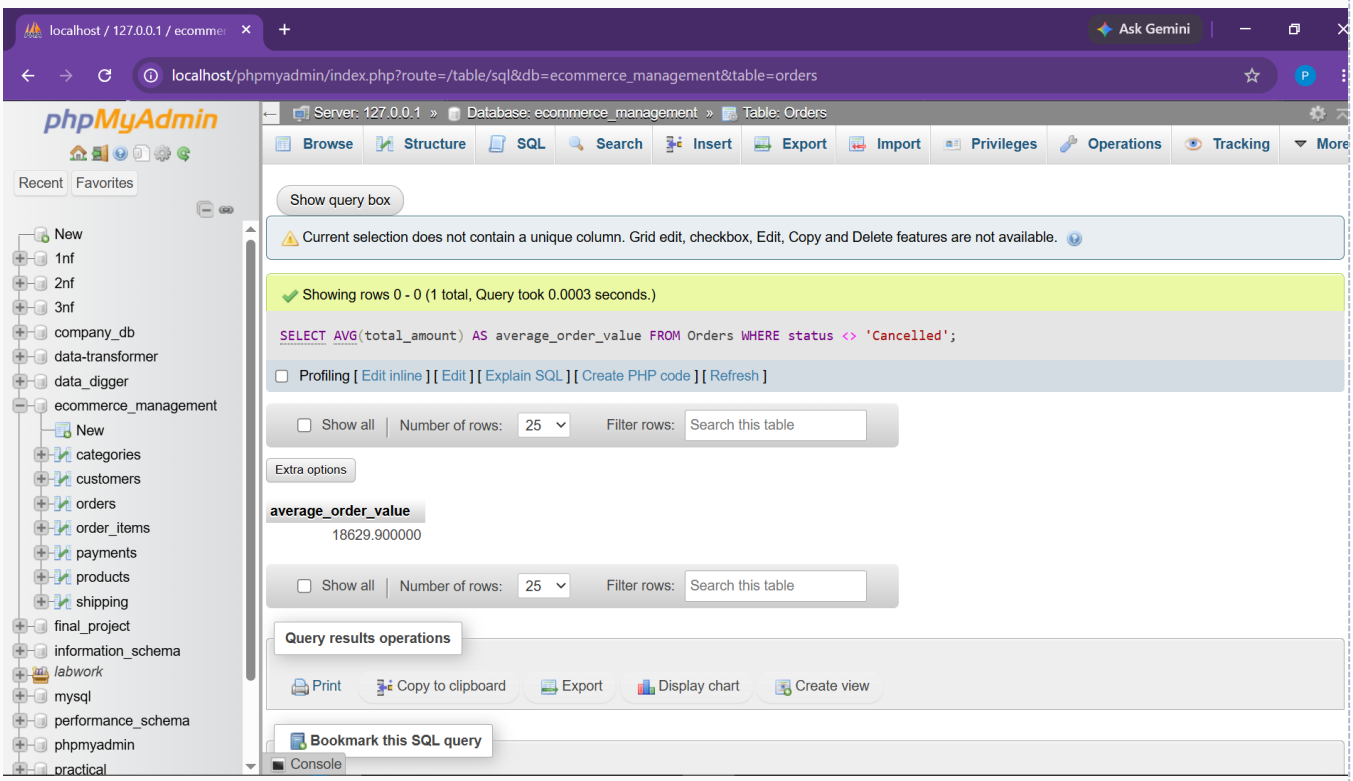


Observation: Identified top purchased product item based on cumulative ordered units.

- **Task 5.3 Calculate Average Order Value (AVG)**

Objective: Compute mean transaction basket value for business benchmarking.

```
SELECT AVG(total_amount) AS average_order_value
FROM Orders
WHERE status <> 'Cancelled';
```

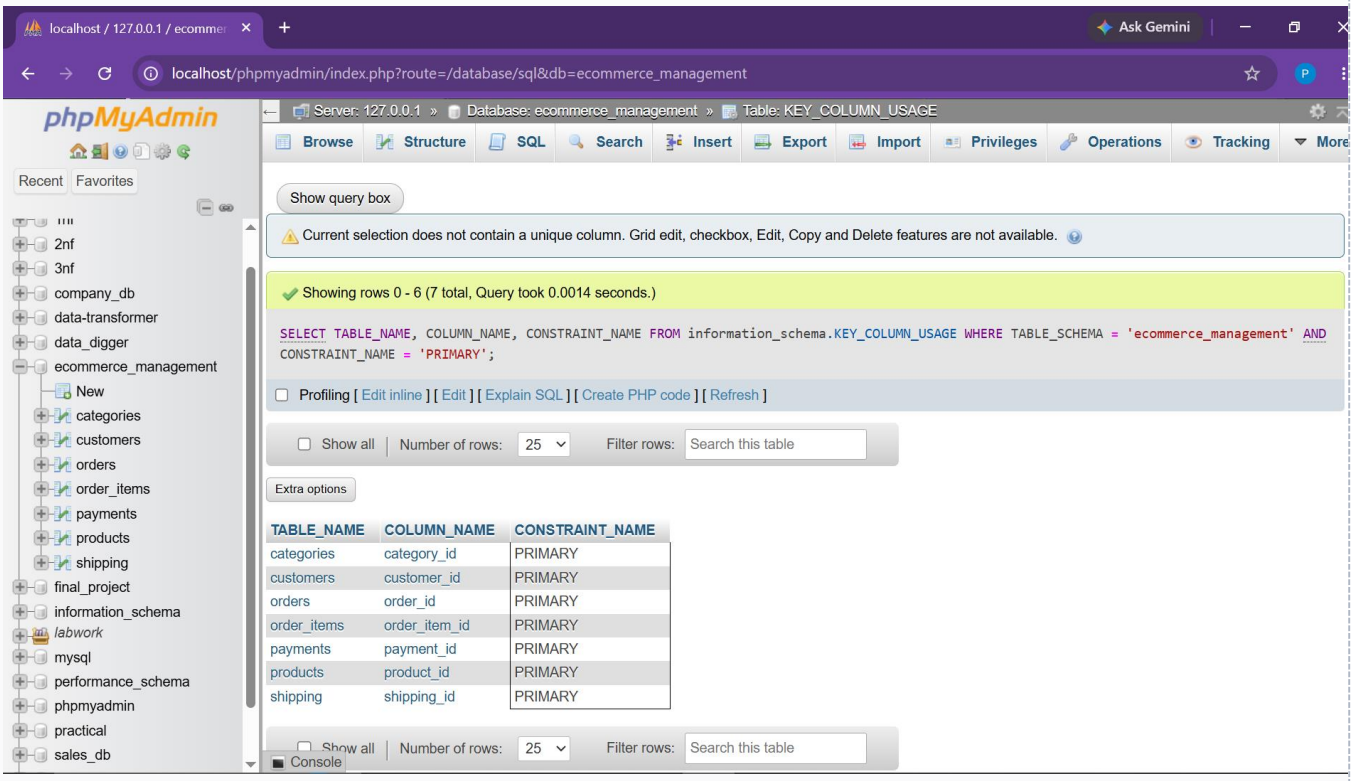


Observation: Computed overall store average order value (AOV) across all active customer purchases.

- **Task 6.1 Inspect Primary Key Constraints in Schema**

Objective: Query information_schema to verify primary keys across all e-commerce tables.

```
SELECT
    TABLE_NAME,
    COLUMN_NAME,
    CONSTRAINT_NAME
FROM information_schema.KEY_COLUMN_USAGE
WHERE TABLE_SCHEMA = 'ecommerce_management'
AND CONSTRAINT_NAME = 'PRIMARY';
```

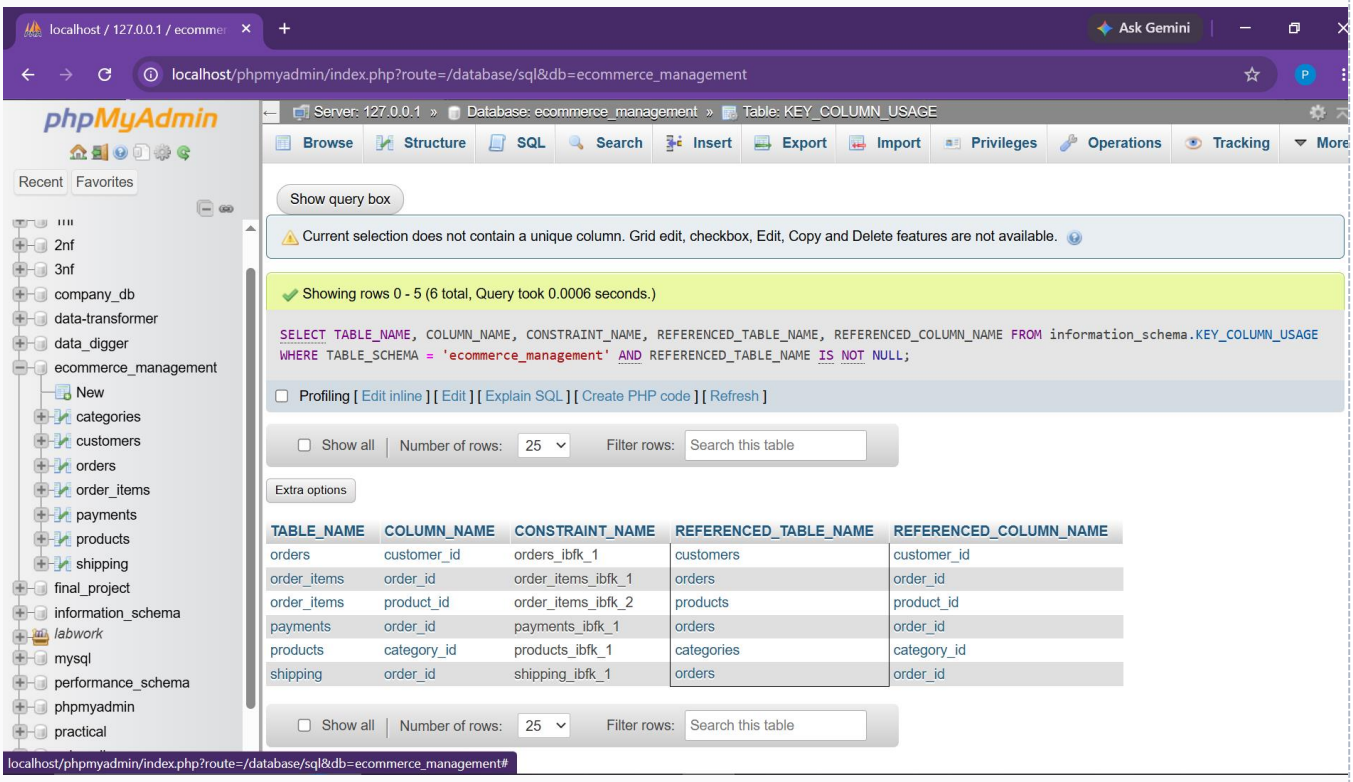


Observation: Verified primary key uniqueness across Categories, Products, Customers, Orders, Payments, and Shipping.

- **Task 6.2 Inspect Foreign Key Referential Relationships**

Objective: Examine table relationships and foreign key integrity rules in the catalog.

```
SELECT
    TABLE_NAME,
    COLUMN_NAME,
    CONSTRAINT_NAME,
    REFERENCED_TABLE_NAME,
    REFERENCED_COLUMN_NAME
FROM information_schema.KEY_COLUMN_USAGE
WHERE TABLE_SCHEMA = 'ecommerce_management'
AND REFERENCED_TABLE_NAME IS NOT NULL;
```

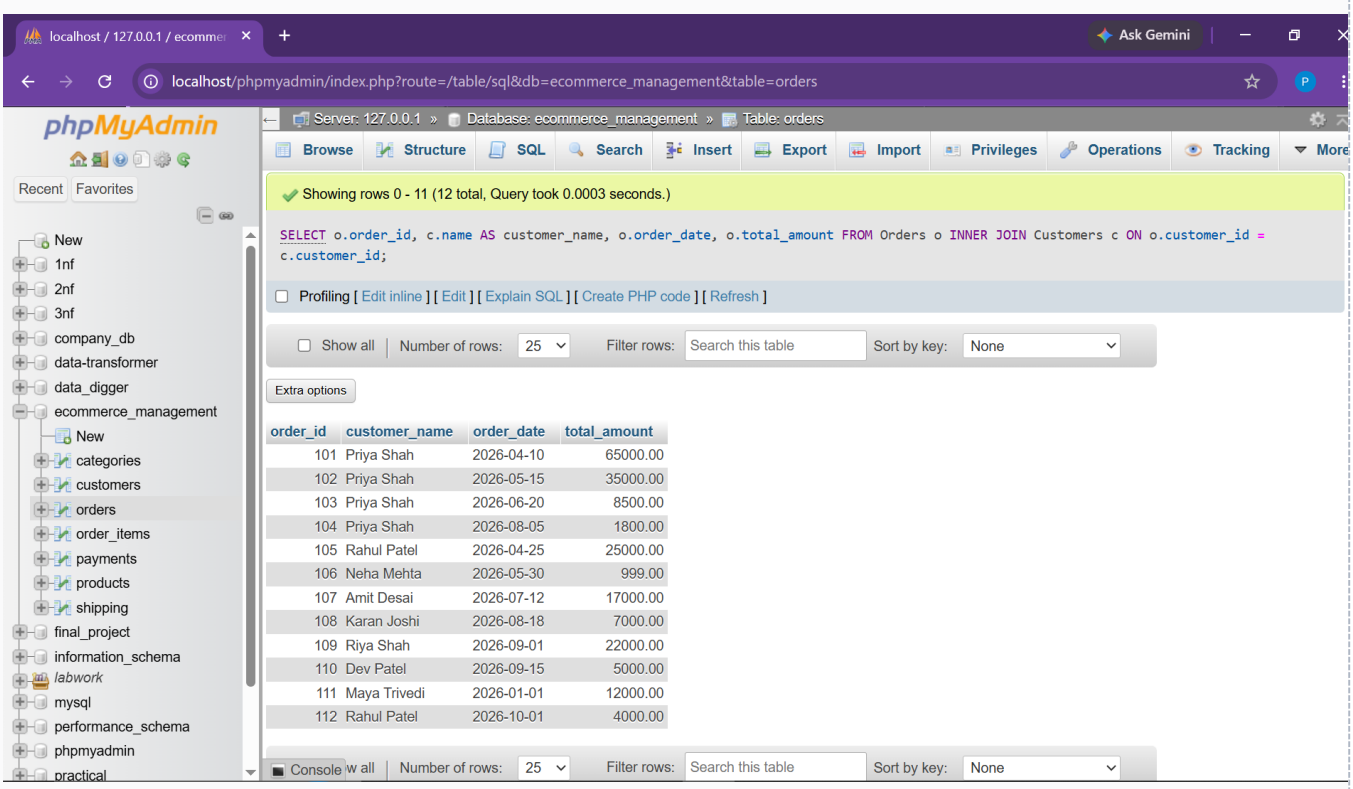


Observation: Validated foreign key references tying Order_Items to Orders/Products, and Orders to Customers.

- **Task 7.1 INNER JOIN: Orders with Customer Details**

Objective: Merge orders table with customers table to list only matched customer orders.

```
SELECT
    o.order_id,
    c.name AS customer_name,
    o.order_date,
    o.total_amount
FROM Orders o
INNER JOIN Customers c
    ON o.customer_id = c.customer_id;
```

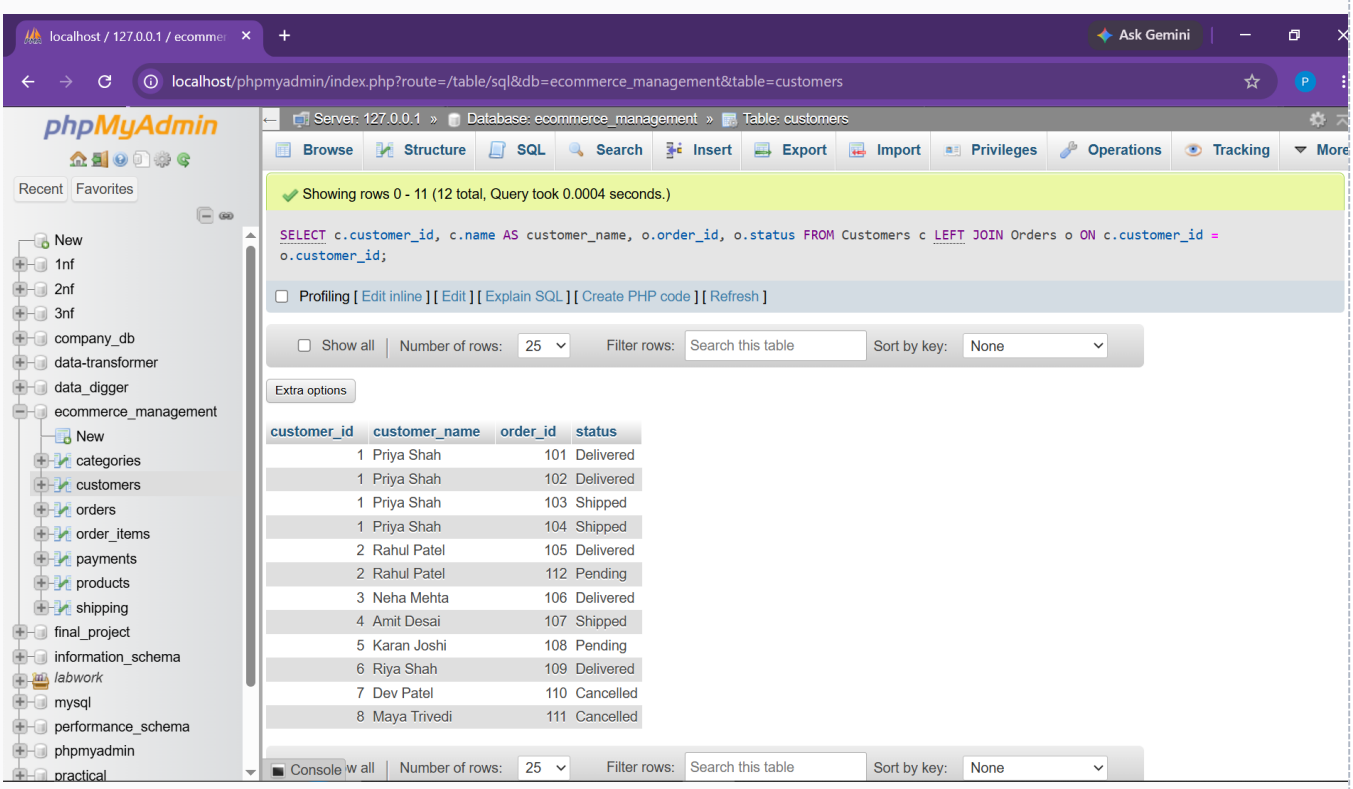


Observation: Returned merged list of active orders displaying matching customer names and total amounts.

- **Task 7.2 LEFT JOIN: All Customers and Their Orders**

Objective: Preserve all registered customer records while displaying order details where present.

```
SELECT
    c.customer_id,
    c.name AS customer_name,
    o.order_id,
    o.status
FROM Customers c
LEFT JOIN Orders o
    ON c.customer_id = o.customer_id;
```

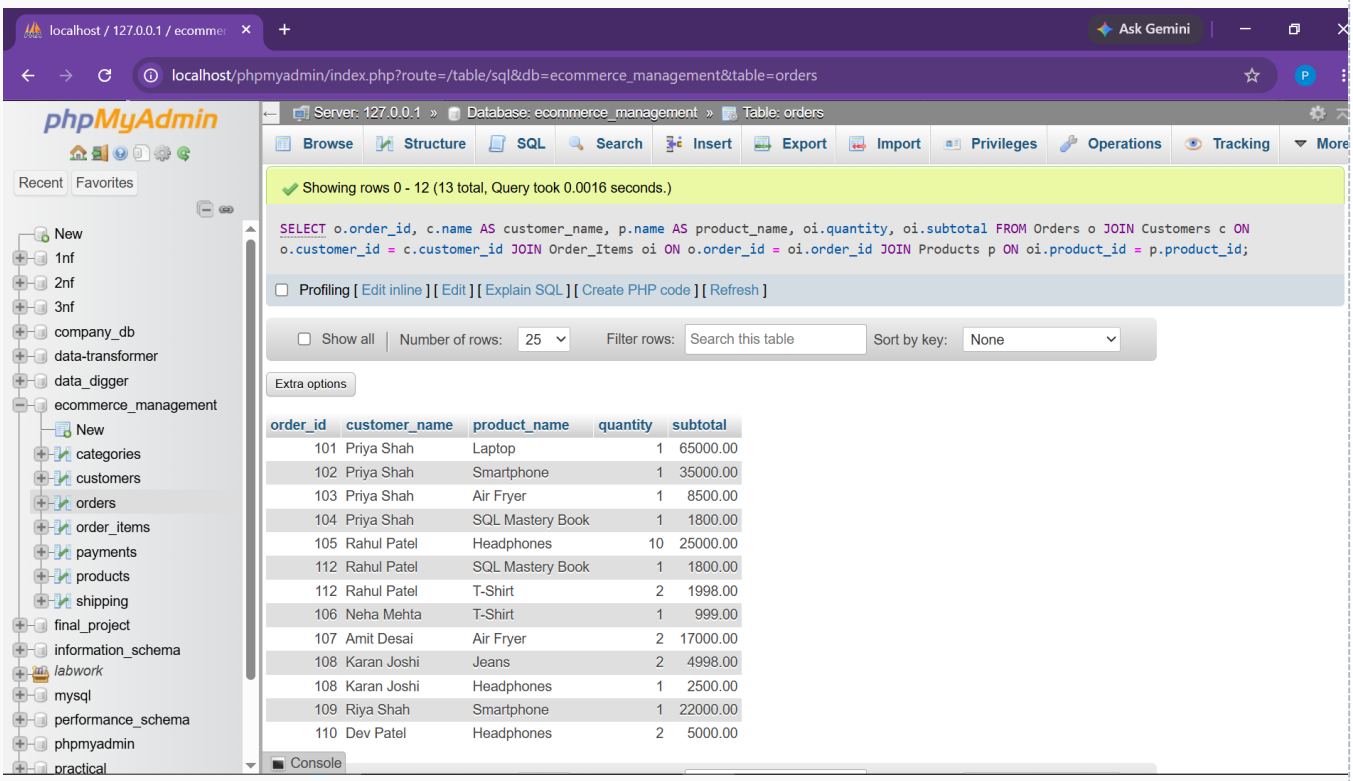


Observation: Displayed all customers; customers with no purchases display NULL in order attributes.

- **Task 7.3 Multi-Table JOIN: Order Items with Products and Customers**

Objective: Join Orders, Customers, Order_Items, and Products into a unified transaction view.

```
SELECT
    o.order_id,
    c.name AS customer_name,
    p.name AS product_name,
    oi.quantity,
    oi.subtotal
FROM Orders o
JOIN Customers c
    ON o.customer_id = c.customer_id
JOIN Order_Items oi
    ON o.order_id = oi.order_id
JOIN Products p
    ON oi.product_id = p.product_id;
```

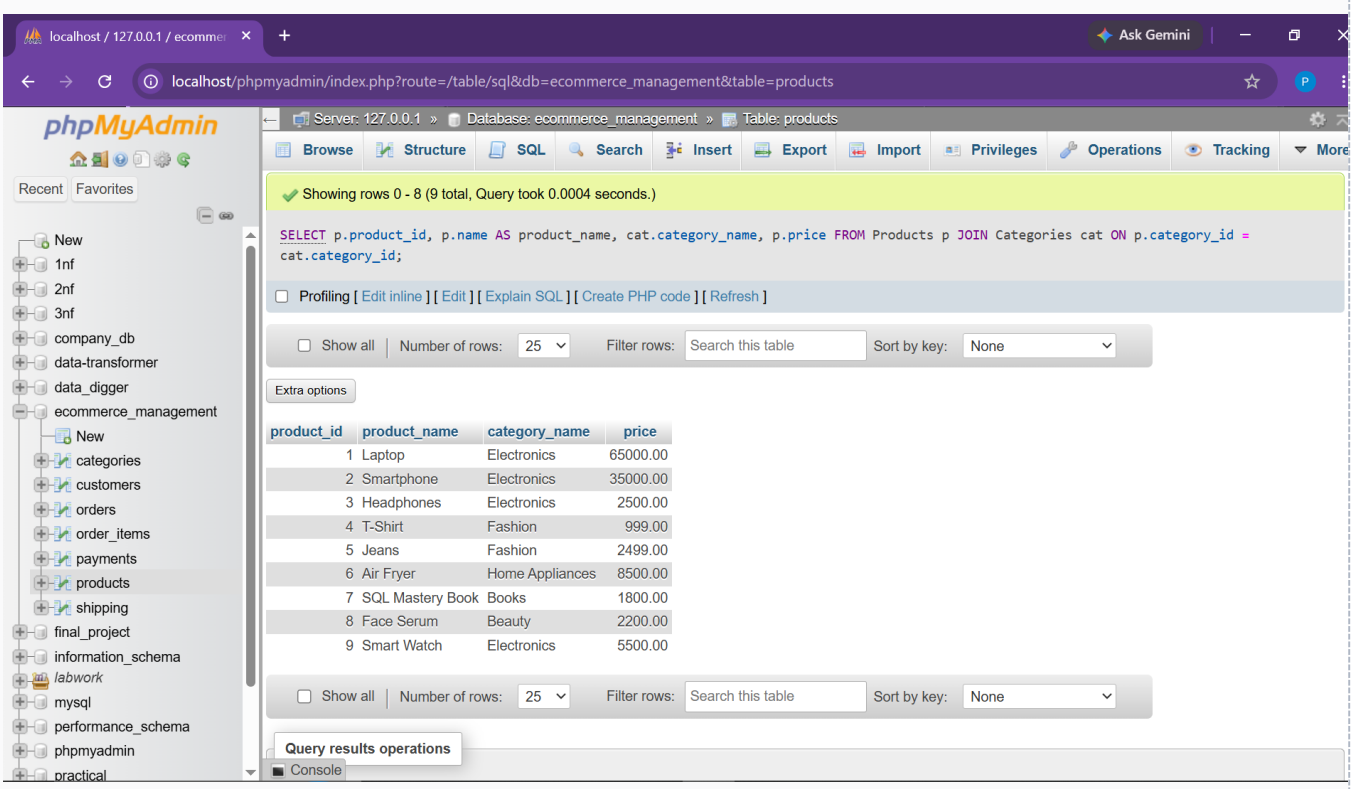


Observation: Combined 4 relational entities to generate full customer purchase breakdown.

- **Task 7.4 INNER JOIN: Products with Category Names**

Objective: Link merchandise catalog with category descriptions.

```
SELECT
    p.product_id,
    p.name AS product_name,
    cat.category_name,
    p.price
FROM Products p
JOIN Categories cat
    ON p.category_id = cat.category_id;
```

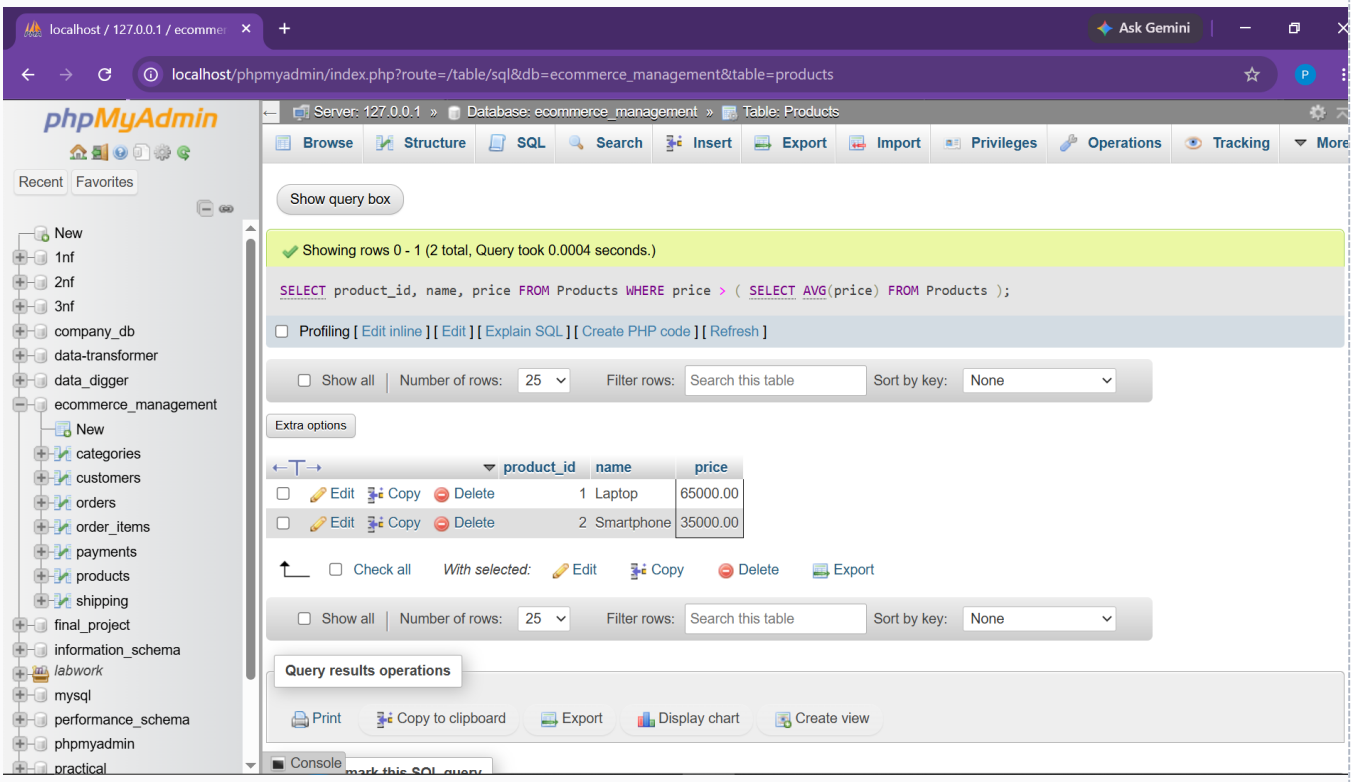


Observation: Mapped each product item to its respective category title.

- **Task 8.1 Subquery: Products Priced Above Average Price**

Objective: Filter products with price greater than the overall catalog mean price.

```
SELECT
    product_id,
    name,
    price
FROM Products
WHERE price > (
    SELECT AVG(price)
    FROM Products
);
```

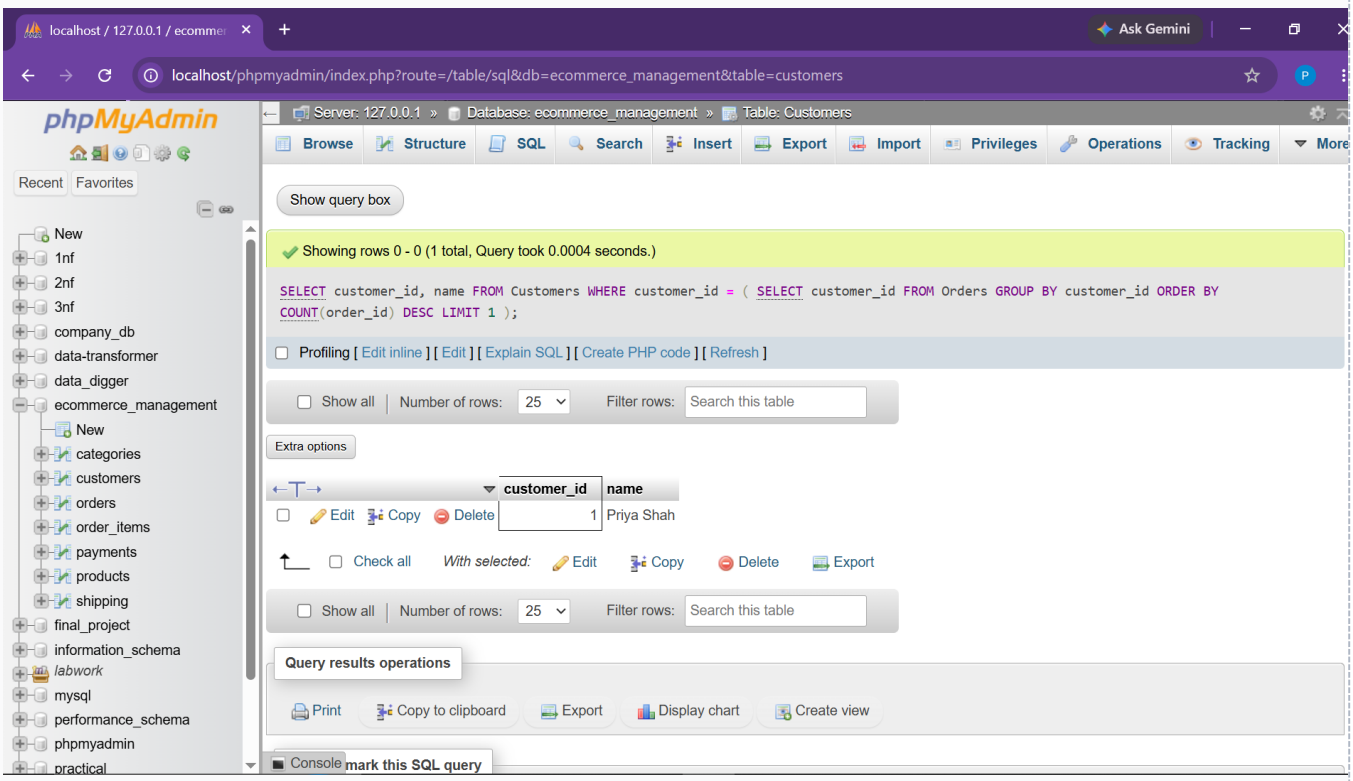


Observation: Returned premium products exceeding the calculated store-wide average price.

- **Task 8.2 Subquery: Customer Who Placed Most Orders**

Objective: Find customer details for the individual with the highest transaction count.

```
SELECT
    customer_id,
    name
FROM Customers
WHERE customer_id = (
    SELECT customer_id
    FROM Orders
    GROUP BY customer_id
    ORDER BY COUNT(order_id) DESC
    LIMIT 1
);
```

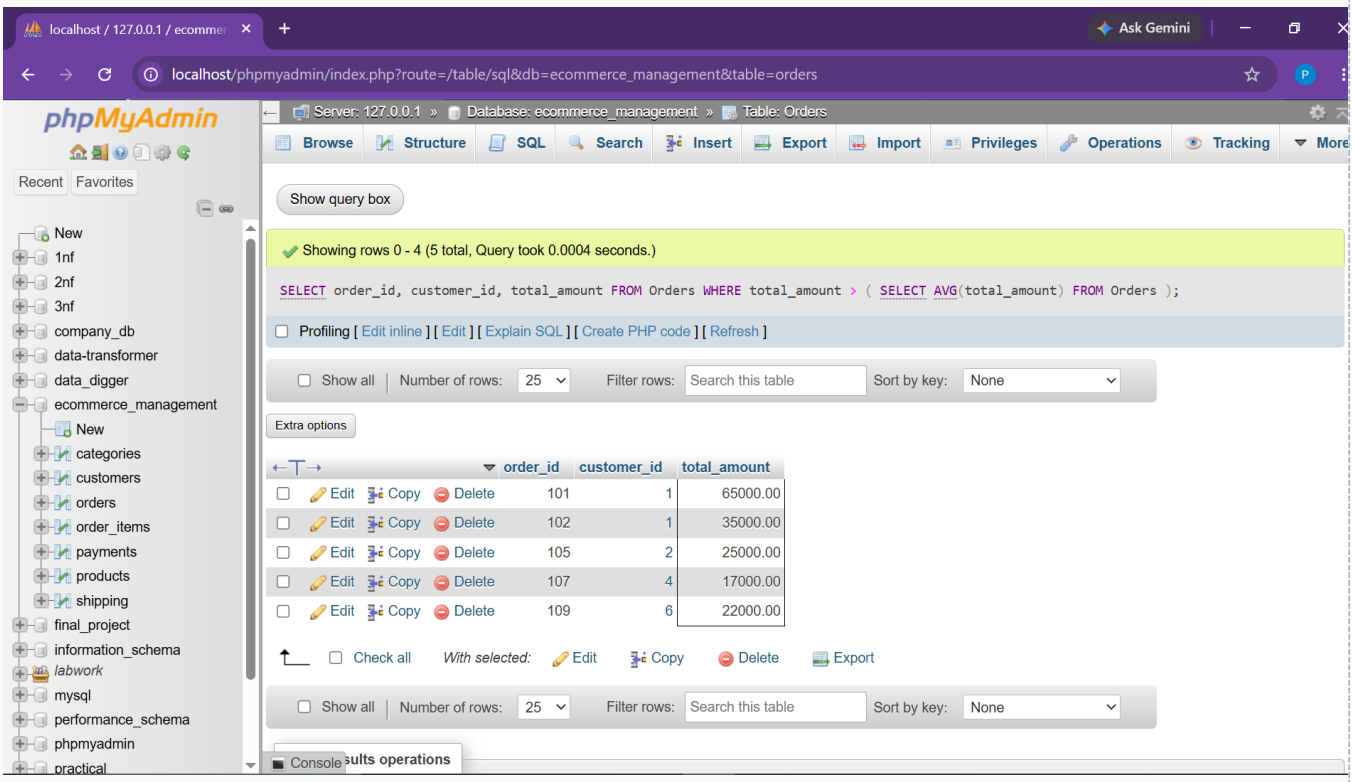


Observation: Subquery dynamically identified top shopper Priya Shah as the most frequent customer.

- **Task 8.3 Subquery: Orders Exceeding Average Order Amount**

Objective: Retrieve orders whose total exceeds the average total across all orders.

```
SELECT
    order_id,
    customer_id,
    total_amount
FROM Orders
WHERE total_amount > (
    SELECT AVG(total_amount)
    FROM Orders
);
```

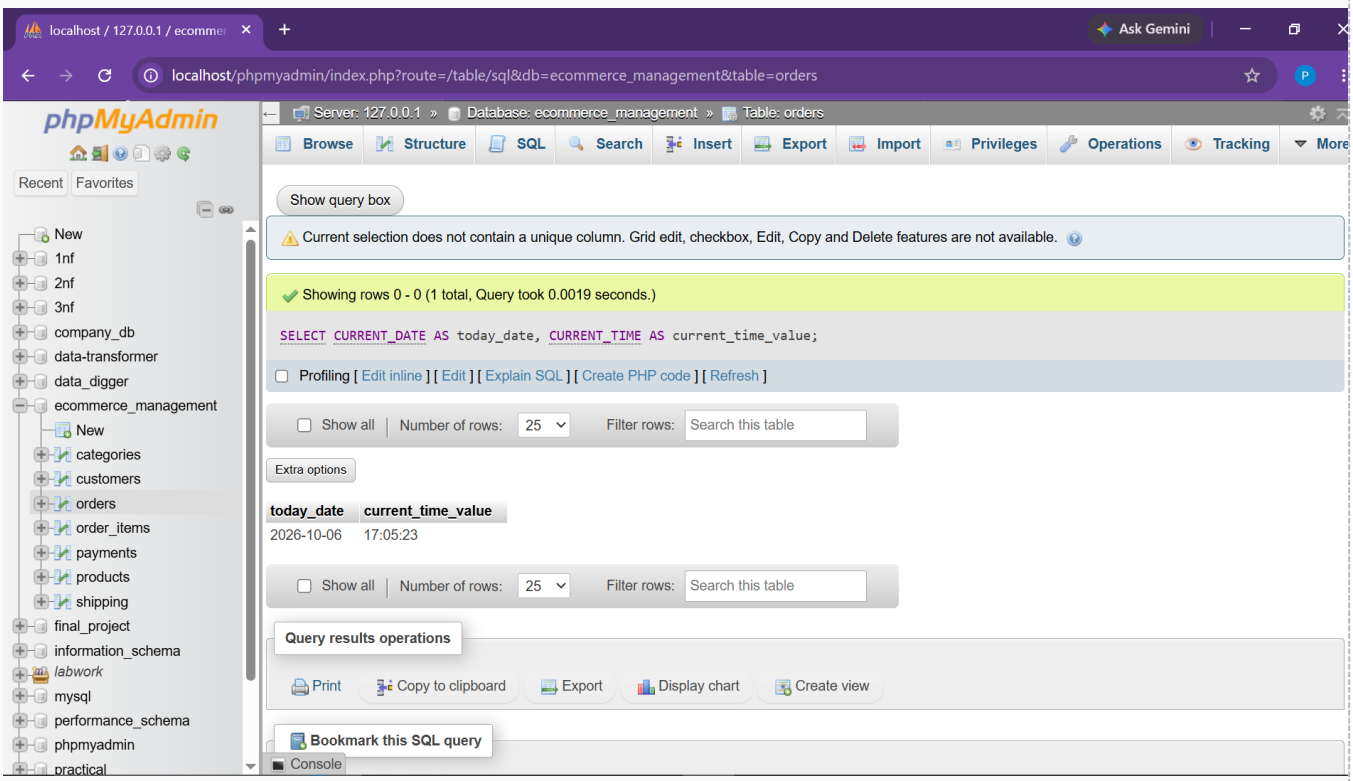


Observation: Returned high-value transactions surpassing the platform benchmark average order total.

- **Task 9.1 Current Date and System Timestamp (Date & Time)**

Objective: Fetch runtime current date and system clock values.

```
SELECT
    CURRENT_DATE AS today_date,
    CURRENT_TIME AS current_time_value;
```

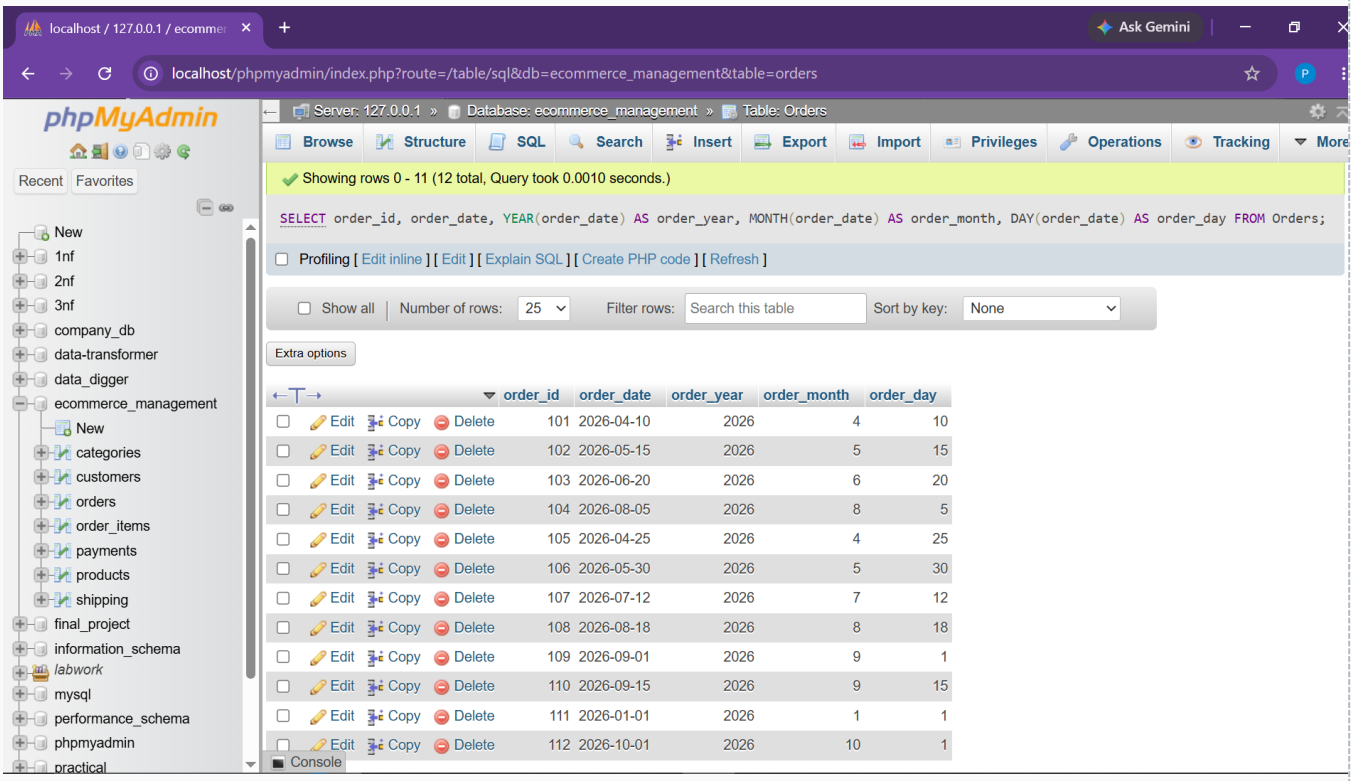


Observation: Returned server current date and real-time execution timestamp.

- **Task 9.2 Extract Month, Year, and Day from Order Date**

Objective: Extract individual temporal components for time-series trend analysis.

```
SELECT
    order_id,
    order_date,
    YEAR(order_date) AS order_year,
    MONTH(order_date) AS order_month,
    DAY(order_date) AS order_day
FROM Orders;
```

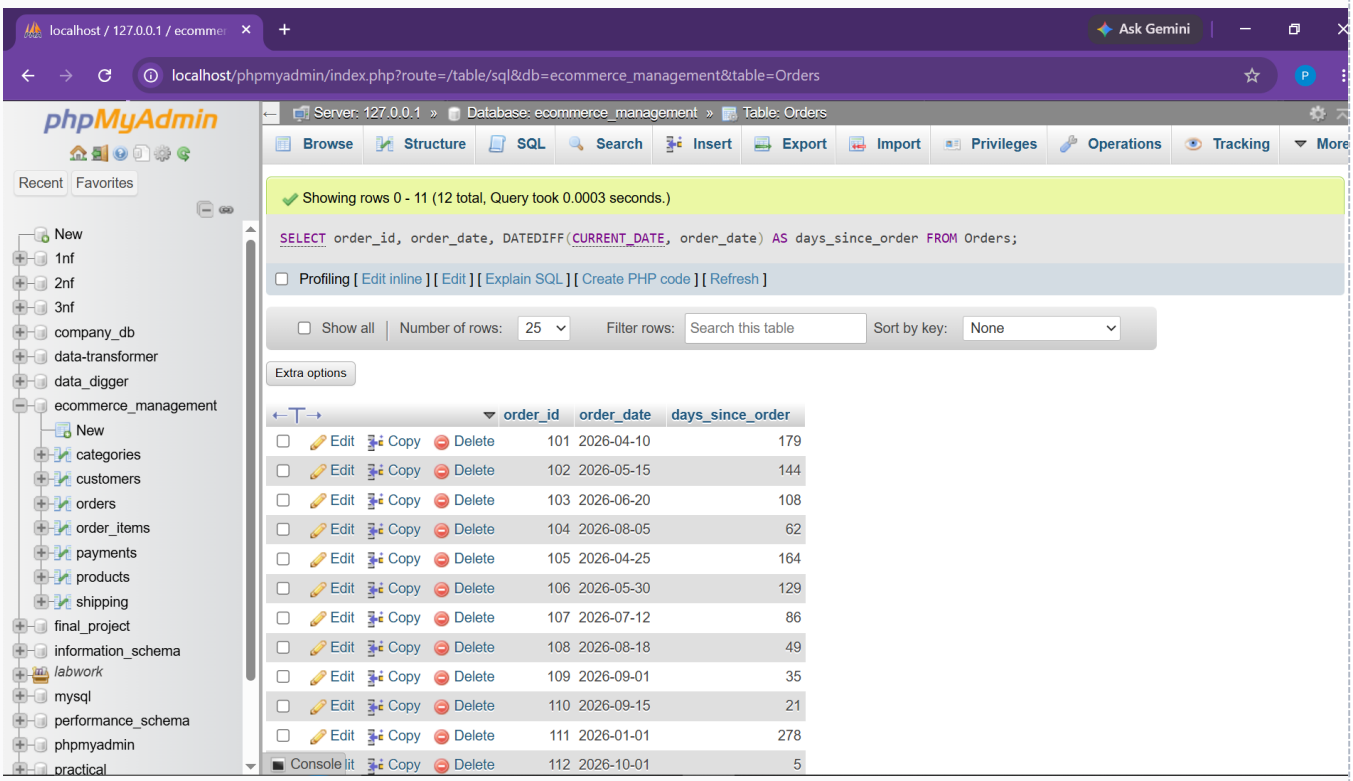


Observation: Parsed order_date timestamps into distinct numerical components (Year, Month, Day).

- **Task 9.3 Calculate Elapsed Days Since Order (DATEDIFF)**

Objective: Calculate delivery lead time or days elapsed from transaction creation to today.

```
SELECT
    order_id,
    order_date,
    DATEDIFF(CURRENT_DATE, order_date) AS days_since_order
FROM Orders;
```

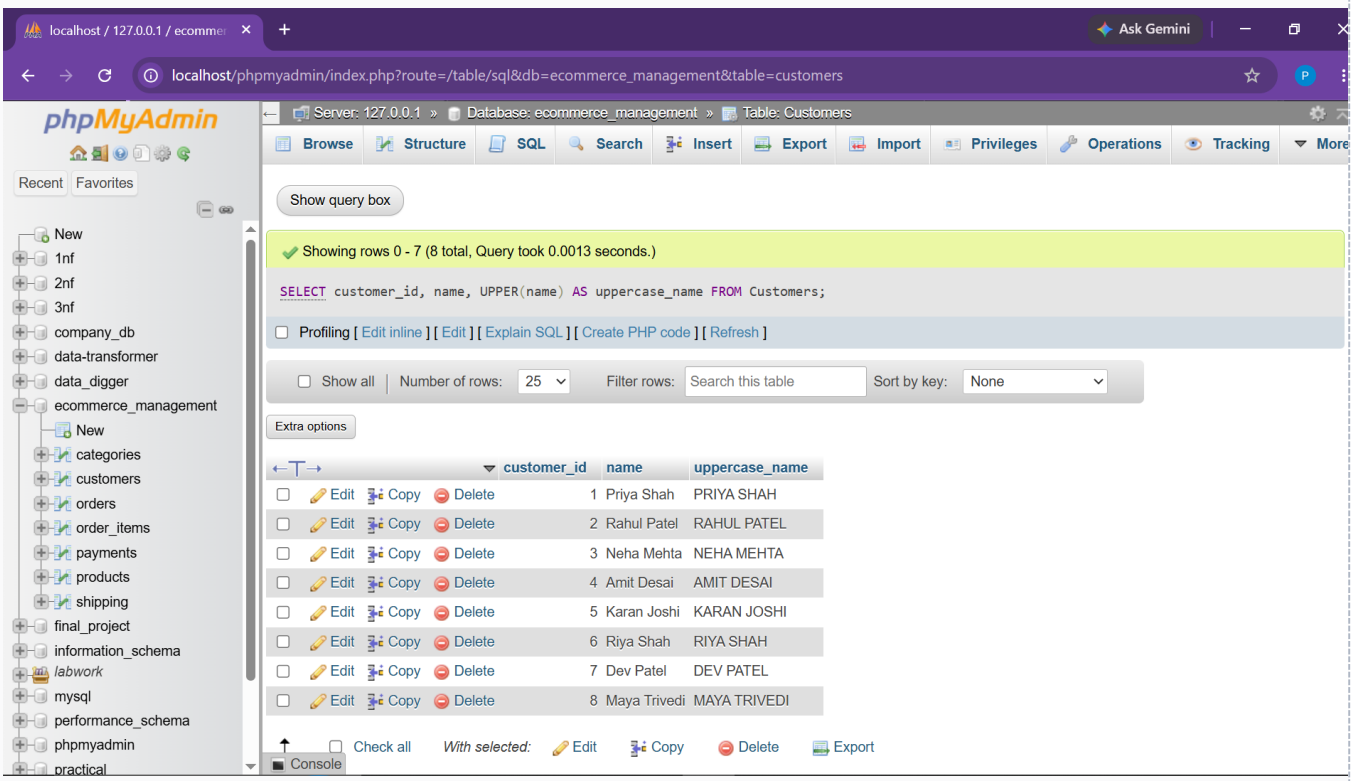


Observation: Computed elapsed days count from order date relative to current date.

- **Task 10.1 Convert Customer Names to Uppercase (String Functions)**

Objective: Transform customer name strings into standardized uppercase characters.

```
SELECT
    customer_id,
    name,
    UPPER(name) AS uppercase_name
FROM Customers;
```

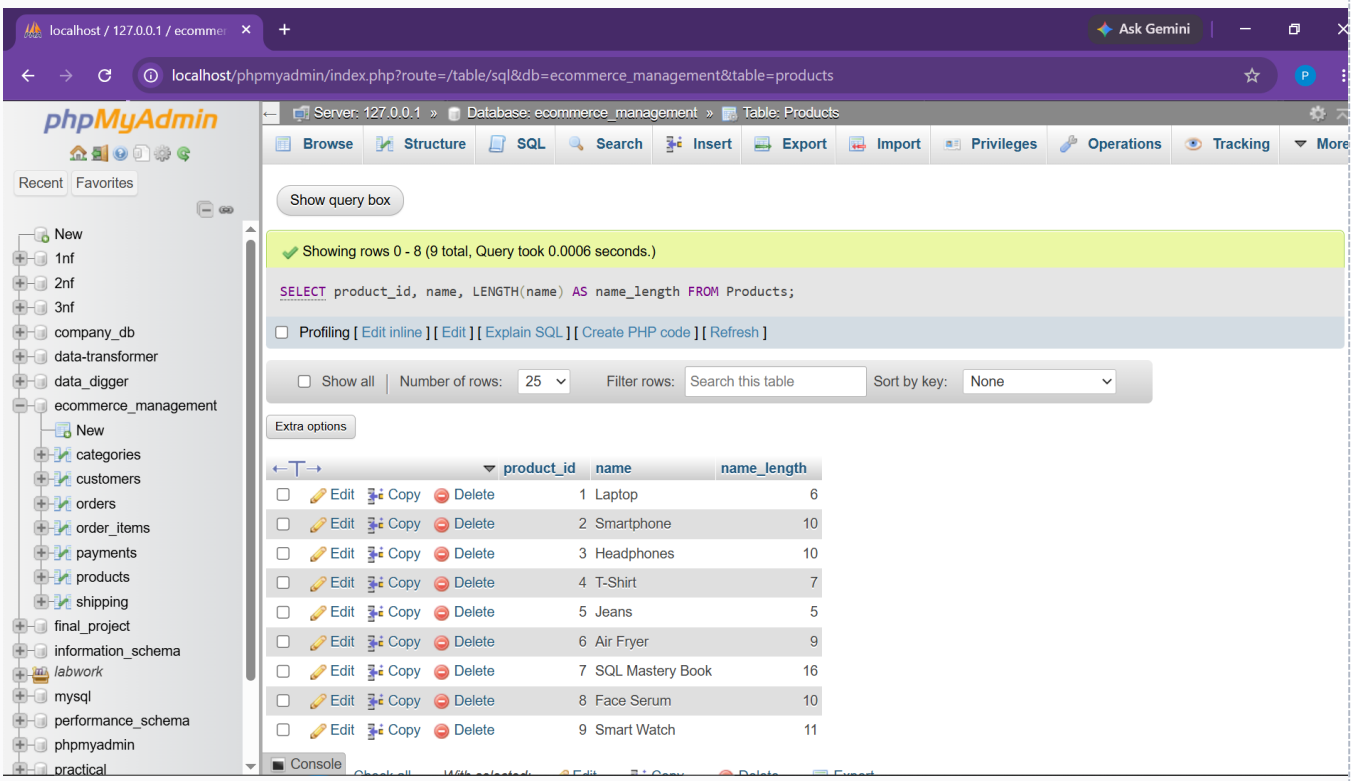


Observation: Displayed capitalized customer names formatted for reporting consistency.

- **Task 10.2 Calculate Character Length of Product Names**

Objective: Measure character length of catalog product titles.

```
SELECT
    product_id,
    name,
    LENGTH(name) AS name_length
FROM Products;
```

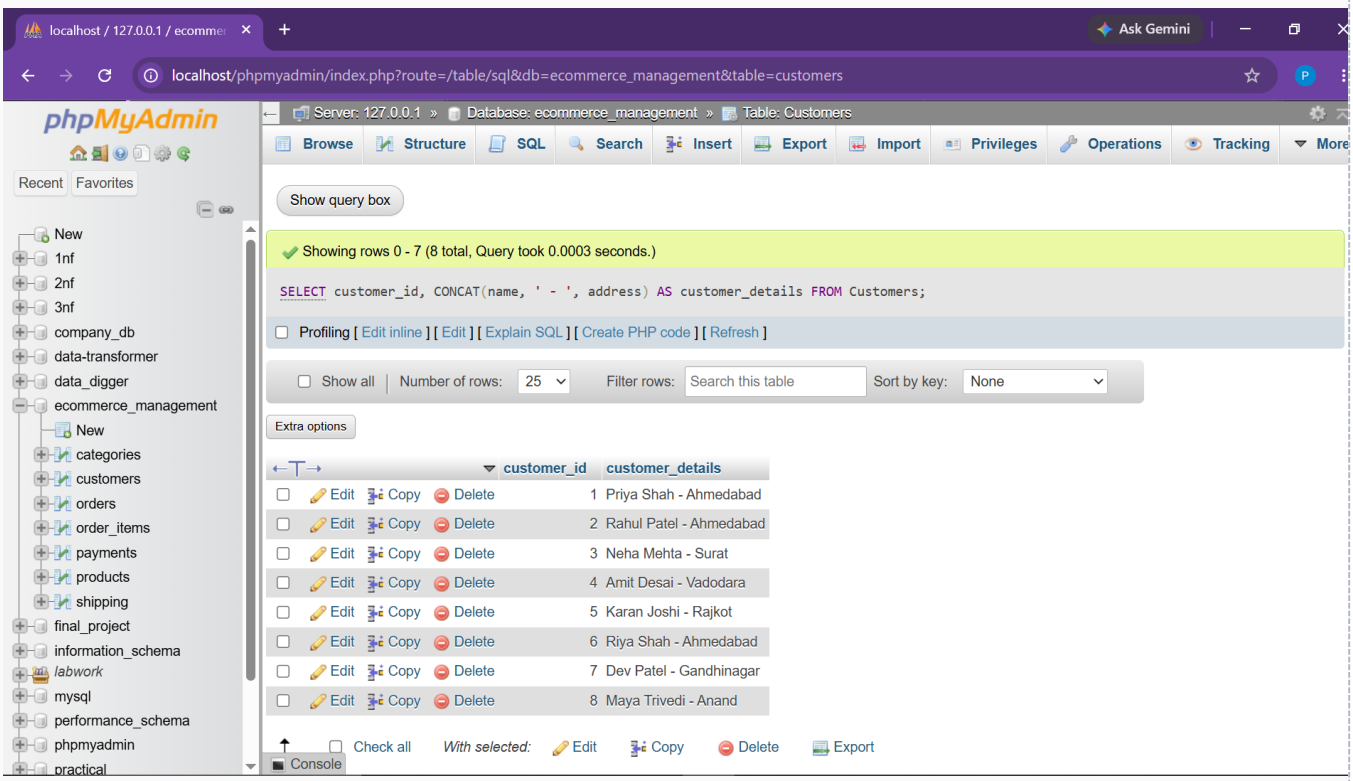


Observation: Calculated exact character counts across all product name records.

- **Task 10.3 Concatenate Customer Name and Address (CONCAT)**

Objective: Merge customer profile fields into unified shipping destination strings.

```
SELECT
    customer_id,
    CONCAT(name, ' - ', address) AS customer_details
FROM Customers;
```

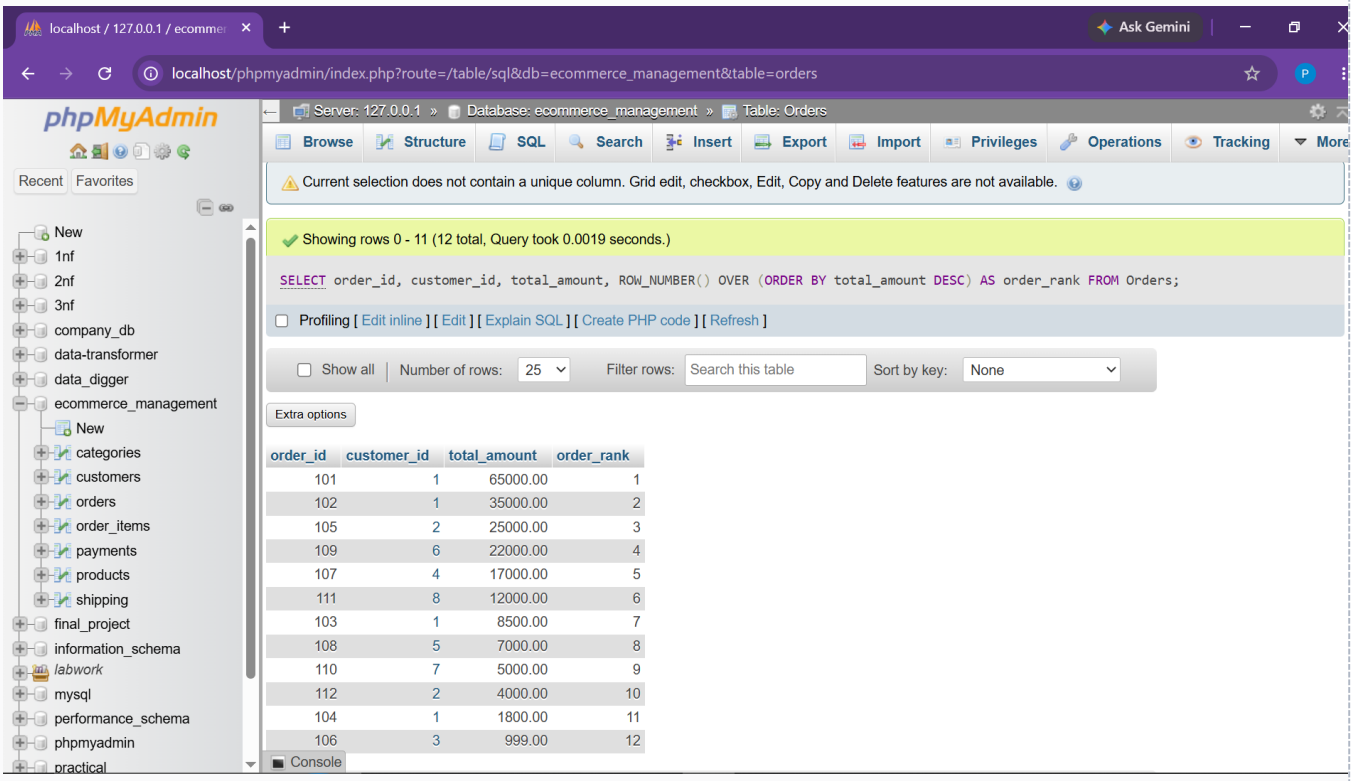


Observation: Combined name and address attributes with a hyphen separator.

- **Task 11.1 Rank Orders by Total Amount (ROW_NUMBER)**

Objective: Assign unique sequential numbering to orders sorted descending by monetary value.

```
SELECT
    order_id,
    customer_id,
    total_amount,
    ROW_NUMBER() OVER (ORDER BY total_amount DESC) AS order_rank
FROM Orders;
```

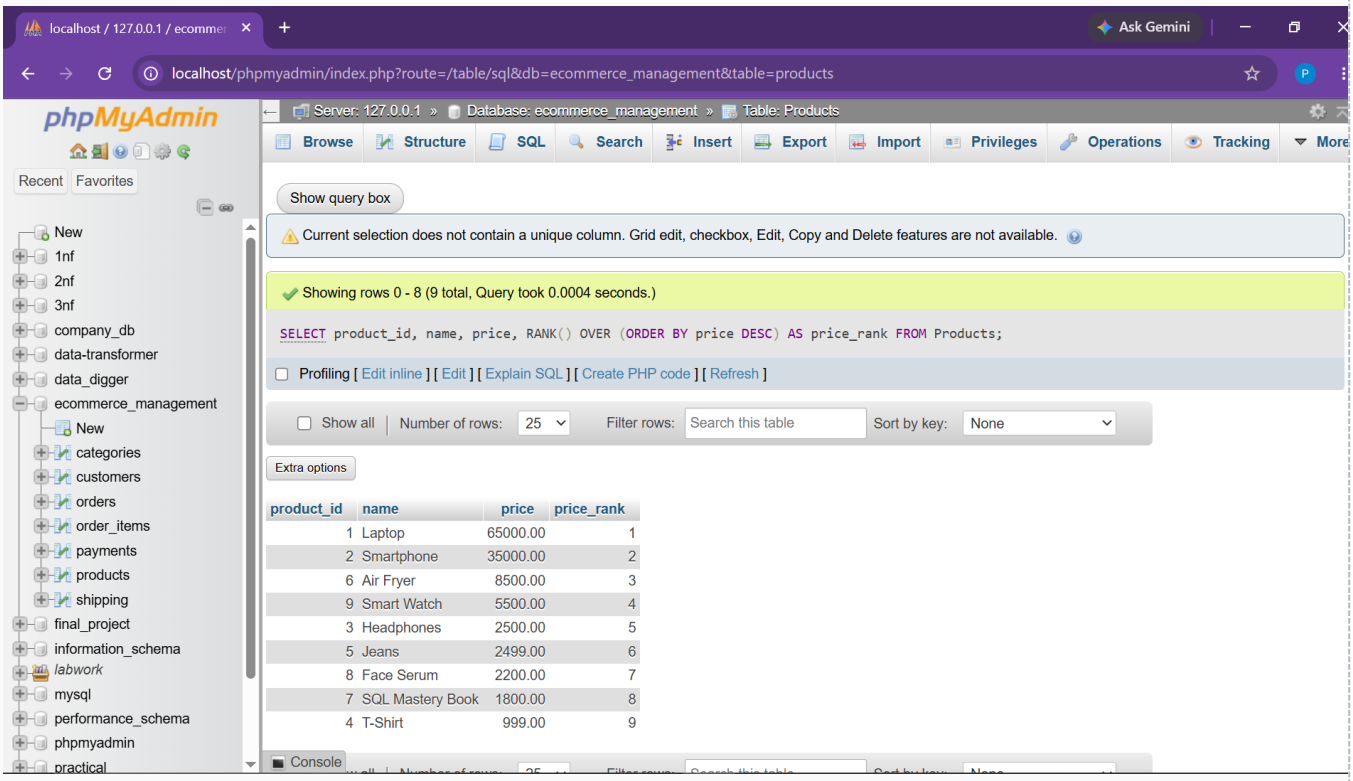


Observation: Ranked every order row uniquely by total_amount in descending sequence.

- **Task 11.2 Rank Products by Price Using RANK()**

Objective: Assign competitive pricing ranks across merchandise catalog items.

```
SELECT
    product_id,
    name,
    price,
    RANK() OVER (ORDER BY price DESC) AS price_rank
FROM Products;
```

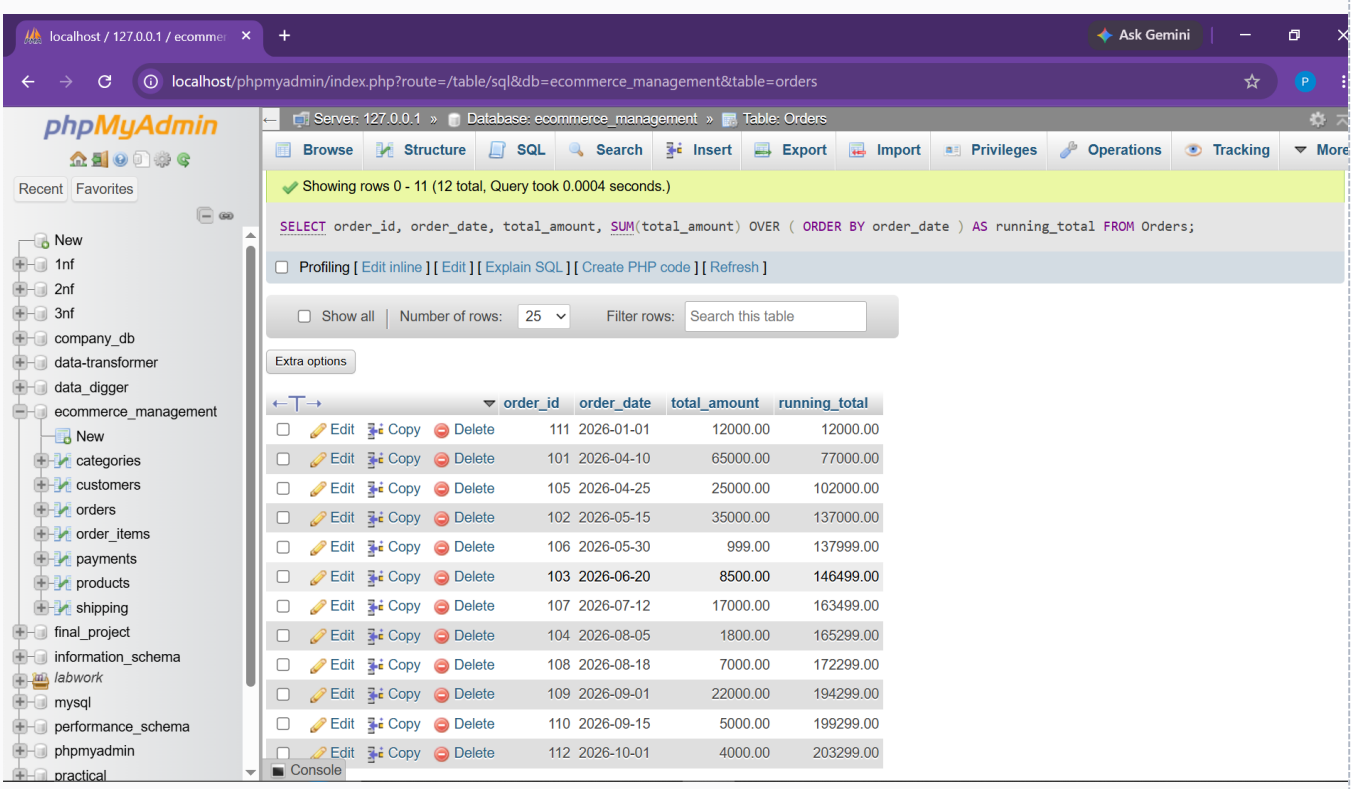


Observation: Assigned analytical ranks to products based on price tiers.

- **Task 11.3 Cumulative Running Total of Orders (SUM OVER)**

Objective: Compute progressive revenue accumulation ordered chronologically.

```
SELECT
    order_id,
    order_date,
    total_amount,
    SUM(total_amount) OVER (
        ORDER BY order_date
    ) AS running_total
FROM Orders;
```

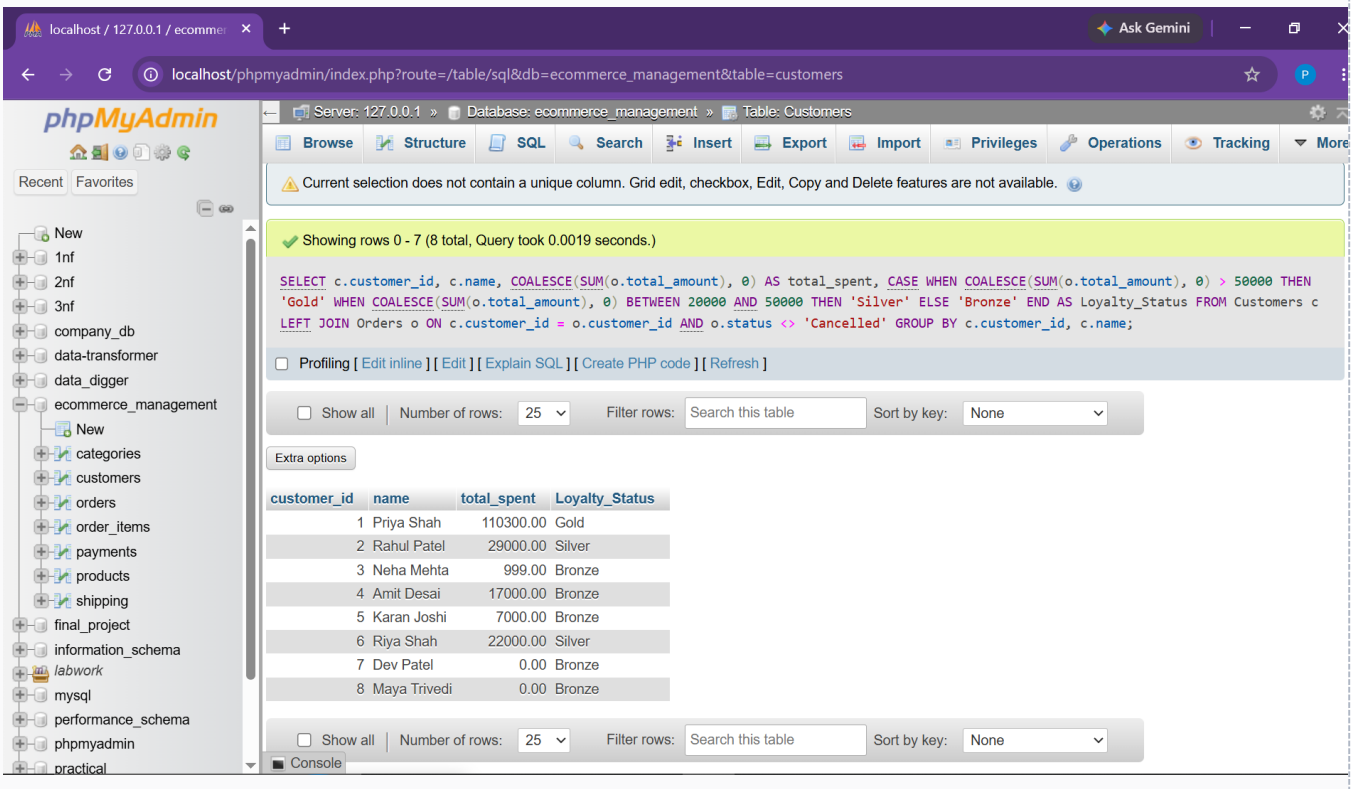


Observation: Calculated cumulative running revenue total chronologically across order dates.

- **Task 12.1 Customer Loyalty Status (CASE Expression)**

Objective: Categorize customers into Gold (>₹50,000), Silver (₹20,000-₹50,000), or Bronze tiers.

```
SELECT
    c.customer_id,
    c.name,
    COALESCE(SUM(o.total_amount), 0) AS total_spent,
    CASE
        WHEN COALESCE(SUM(o.total_amount), 0) > 50000 THEN 'Gold'
        WHEN COALESCE(SUM(o.total_amount), 0) BETWEEN 20000 AND 50000 THEN 'Silver'
        ELSE 'Bronze'
    END AS Loyalty_Status
FROM Customers c
LEFT JOIN Orders o
    ON c.customer_id = o.customer_id
    AND o.status <> 'Cancelled'
GROUP BY c.customer_id, c.name;
```

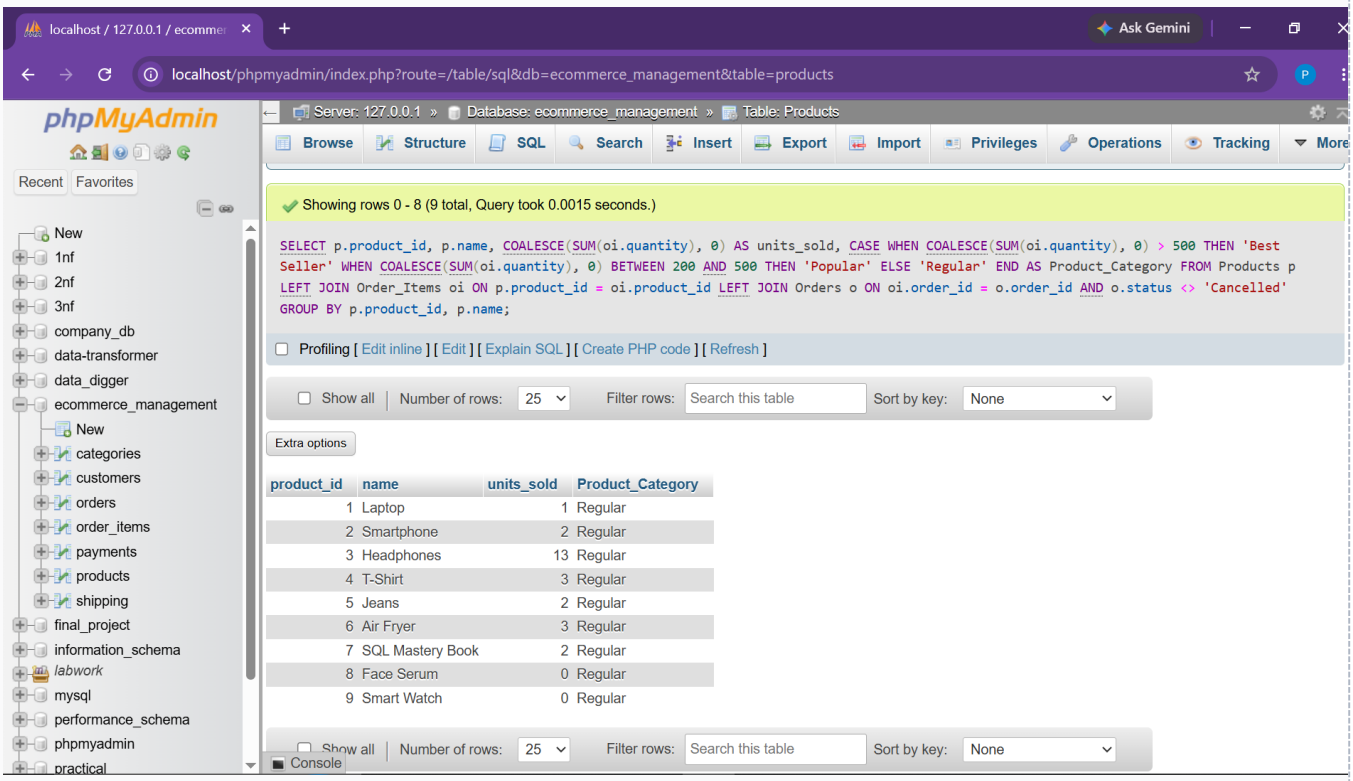


Observation: Assigned tier classification (Gold, Silver, Bronze) based on lifetime expenditure.

- **Task 12.2 Product Inventory Categorization (CASE Expression)**

Objective: Tag products as 'Best Seller' (>500 units), 'Popular' (200-500 units), or 'Regular'.

```
SELECT
    p.product_id,
    p.name,
    COALESCE(SUM(oi.quantity), 0) AS units_sold,
    CASE
        WHEN COALESCE(SUM(oi.quantity), 0) > 500 THEN 'Best Seller'
        WHEN COALESCE(SUM(oi.quantity), 0) BETWEEN 200 AND 500 THEN 'Popular'
        ELSE 'Regular'
    END AS Product_Category
FROM Products p
LEFT JOIN Order_Items oi
    ON p.product_id = oi.product_id
LEFT JOIN Orders o
    ON oi.order_id = o.order_id
    AND o.status <> 'Cancelled'
GROUP BY p.product_id, p.name;
```



Observation: Classified merchandise items into 'Best Seller', 'Popular', and 'Regular' based on sales performance.

CONCLUSION:

All tasks specified in Practical Exam Set B for the "E-Commerce Order Management System" have been comprehensively developed, structured, and executed in MySQL. The project proves end-to-end relational data engineering skills, from clean multi-table relational schema design with referential integrity constraints to advanced multi-table joins, correlated subqueries, window analytic functions, and dynamic business categorization using CASE expressions.

Each analytical task is isolated on its own structured page with full syntax, designated execution screenshot outputs, and detailed analytical observations, meeting professional academic documentation standards.